Session 2 speaking script: Vocabulary Types and the Standard Library

How to use this script

- **Bold lines** are the must-say sentences. If you say nothing else on a slide, say those.
- Plain paragraphs are the talk track, written to be read aloud. Paraphrase freely once rehearsed.
- `(pause)` and `(beat)` are deliberate stops. A pause is two full seconds of silence; a beat is one. You will want to skip them. Don't.
- `>> DO:` lines are actions: terminal commands, editor moves, Compiler Explorer edits, with expected output and a recovery line.
- `>> ASK:` lines are questions to the room, with the answer you expect and what to say after. Wait for Teams latency: count to five before you answer it yourself.
- `>> IF AHEAD` is 60 to 120 seconds of real extra depth; `>> IF BEHIND` is the one sentence to say instead of the slide.

Pace plan for a fast speaker

Four blocks: about twenty minutes of slides, then a ten-minute lab, four times. The lab is the checkpoint; if the lecture runs long, the lab still starts on time and the slides you did not reach are covered in the lab debrief or skipped (they are all in the deck's Further reading appendix for the PDF).

Checkpoint	Target clock
1. Title	0:00
6. `std::string_view` (block 1, Views)	0:07
14. Lab 2.1 launched	0:20
15. `std::optional` (block 2, Maybe, either, anything)	0:30
27. Lab 2.2 launched	0:50
28. The error-handling landscape (block 3, `expected` and formatting)	1:00
37. `std::format`	1:12
44. Lab 2.3 launched	1:20
45. C++17 tour, 1 (block 4, Library tour and the checklist)	1:30
57. Lab 2.4 launched	1:44
58. Exercise (at home), 59. Takeaway	1:54
Done, with three minutes of slack	1:57

If you hit a lab launch more than 2 minutes early, use the IF AHEAD material in the slide before it rather than starting the lab early; people need the clock to be predictable. If you are late, cut the slide before the takeaway, never the lab.

Slow down deliberately on these four:

- **Slide 8, the dangling rule.** It is the entire cost of every view type today. If they do not internalize "a view is a reference", every later slide on `string_view` and `span` is a footgun handed out for free. Lab 2.1's TODO 3 tests exactly this.
- **Slide 22, `std::visit` and `overloaded`.** Two lines of variadic inheritance plus CTAD. Half the room has never seen `using Fs::operator()...;`. Read it slowly, piece by piece. Lab 2.2's TODO 3 has them write it.
- **Slide 32, monadic `expected`.** The type changes at every step of the chain. Say the type out loud at each line or people lose the thread.
- **Slide 41, `std::formatter`.** `parse` and `format`, the context template. Lab 2.3's TODO 3 and the at-home task 4 are where people get stuck alone.

Before class checklist

- Build with GCC 14 and run the tests, labs included: `cmake -S . -B build -G Ninja && cmake --build build && ctest --test-dir build -R "s02|lab_s02" --output-on-failure`. Optional: a Clang 18 + libc++ build (`CXX=clang++ cmake -S . -B build-clang -G Ninja && cmake --build build-clang`) for range formatting and `mdspan`.
- Run every s02 demo binary once so nothing surprises you: `for b in build/demos/s02/demo_s02_*; do echo "== $b"; $b; done`.
- Open the four lab starters on Compiler Explorer in four tabs, from the links on slides 14, 27, 44 and 57, and do each lab yourself once: `lab1_views`, `lab2_optional_variant`, `lab3_expected_format`, `lab4_interface_checklist`. Keep the four solution links ready to paste at the seven-minute mark of each lab. The whole list, with links, is in `handouts/compiler-explorer-links.md` under "The in-class labs".
- Demo files to have open as editor tabs on the right, in deck order: `string_view_basics.cpp`, `string_view_dangling.cpp`, `span_basics.cpp`, `span_bytes.cpp`, `optional_basics.cpp`, `optional_monadic.cpp`, `variant_visit.cpp`, `variant_messages.cpp`, `any.cpp`, `expected_basics.cpp`, `expected_pipeline.cpp`, `format_specs.cpp`, `print.cpp`, `formatter_custom.cpp`, then the `tour_*.cpp` files. (The appendix slides' demos, `string_view_not_cstring.cpp`, `transparent_compare.cpp`, `expected_void.cpp`, `formatter_spec.cpp`, `format_ranges.cpp`, `format_to_buffer.cpp`, only if you expect to use the IF AHEAD material.)
- Compiler Explorer tabs to preload besides the labs (Clang 18 is the safest target for the error demos):
  1. `string_view_dangling.cpp` with `-std=c++23 -Wall -Wextra -Werror -DSHOW_ERRORS`, Clang 18.
  2. `optional_monadic.cpp` with the trap line uncommented (call it on a local `std::optional` and on a temporary), GCC 14.

3. A three-liner: `#include <format>` and `auto s = std::format("{:d}", 3.14);`, GCC 14, `-std=c++23`.

- Have the exercise README ready to paste for the at-home launch: `exercises/s02-vocabulary-types/README.md`. Confirm `ctest --test-dir build -R s02_report_identical` passes on the starter as-is.
- Have the handouts path ready to paste in chat: `handouts/toolchain-support-matrix.md`, `handouts/cheat-sheet-vocabulary-types.md`, `handouts/compiler-explorer-links.md`.

The script

1. The Evolution of C++, Session 2: Vocabulary Types and the Standard Library · target 0:00, ~1 min

Welcome back. Last session was the language: syntax that makes everyday code shorter and safer. Today is the library, and specifically the types you put in function signatures.

Every type today answers a question that a C++11 interface left to a comment: maybe? which one? did it fail, and why? who owns this?

(pause)

If you only take one session of this course back to your codebase on Monday, it is probably this one. These are the types you will use every day, in every header, starting the next time you write a function signature.

Let's look at how the two hours break down.

2. Agenda · target 0:01, ~1 min

Four blocks. **Twenty minutes of me, ten minutes of you, four times.** Block one is views, `string_view` and `span`. Block two is `optional`, `variant`, and `any`. Block three is `std::expected`, which is the one I most want you to leave with, and formatting. Block four is a fast library tour and the interface checklist.

(beat)

Each lab is one file on Compiler Explorer with three TODOs. The slide that launches it has the link; you need nothing installed. If you built the repo, the same files are under `labs/`.

The telemetry exercise is at-home work this time. Its first three tasks are today's labs applied to the real program, so you will have done them once already. **Last session was too much of me talking. This one is built so you type every half hour.**

Before the new material, a quick look back at what tripped people up last time.

3. Session 1 recap · target 0:02, ~1.5 min

Here's what the Session 1 solution looks like. Same telemetry program, same output, but six comparison operators collapsed into one spaceship, `using` instead of `typedef`, structured

bindings, `[[nodiscard]]` on everything that returns something you shouldn't ignore, `from_chars` instead of `atof`, and `inline constexpr` constants instead of macros.

Three places people got stuck.

First: leaving one old `operator<` declaration in the header next to the new `<=>`. The compiler now has a hand-written `<` and a rewritten one from the spaceship, and it can't pick. Ambiguous overload. The fix is to delete the old one. When you migrate, migrate all the way.

Second: trying to bind a structured binding into existing variables. You can't. **A structured binding always declares new names.** If you want to assign into existing variables, that's `std::tie`, and it's still fine to use.

(pause)

Third, and this is the one I want to spend a minute on: `from_chars` for `double` is missing on libc++ 18. So code that compiled on GCC broke on Clang. The fix was not `#if __clang_major__ < 20`. The fix was the feature-test macro, `__cpp_lib_to_chars`.

Test the feature, not the compiler version.

Since C++20, the `<version>` header defines a `__cpp_lib_` macro for every library feature. Your code can ask "do I have this?" instead of guessing from a version number. That matters a lot in our world, where the certified toolchain lags the current one by years and the same code often builds on two compilers. It comes back at the end of today on the support matrix slide, because today's features have the most uneven support of the whole course.

> IF AHEAD: The macros carry a date value, not just defined-or-not. `__cpp_lib_optional` is `201606L` for the C++17 version and `202110L` once the monadic operations are in. So you can write `#if __cpp_lib_optional >= 202110L` to gate on the C++23 `and_then`. The values are listed on cppreference's feature-test page. One review heuristic: any `#if` on `__GNUC__` or `__clang_major__` that's really asking about a library feature should be a `__cpp_lib_` check instead, because the library and the compiler are not the same thing. Clang with libstdc++ and Clang with libc++ give different answers.
>
> IF BEHIND: Session 1's solution is today's starter; the lesson from its rough edges is to test features with `__cpp_lib_` macros, not compiler versions.

That starter still has some C++11-shaped interfaces in it. Let's name them.

4. What is a vocabulary type? · target 0:03, ~1.5 min

A vocabulary type is a type that appears in interfaces so that caller and callee agree on meaning without reading a comment.

The word "vocabulary" matters. `int` is vocabulary. `std::string` is vocabulary. Everybody's code uses the same one, so you can pass it across library boundaries without adapters. Today's types are the same idea for concepts we've always had but never had a standard word for.

Look at the table. These are five signatures from the Session 1 solution, the thing that is supposedly already modern.

Row one: `bool parse(const std::string&, Record* out, ParseError* err)`. What it really means is "a Record, or a reason". But the signature says "a bool and two pointers", and you need the comment to know which pointer gets written when.

Row two: `const SensorConfig* find_sensor`. Meaning: "maybe a config". The pointer says that, but it also says "and it lives somewhere else, go figure out how long".

Row three, my favorite: `std::pair<double,double> value_range(const std::vector<Record>&)`. Min and max, **if not empty**. That "if not empty" is a precondition living in a comment. Call it with an empty vector and you get whatever the implementation happens to do.

(pause)

Row four: `crc16(const unsigned char*, std::size_t)`. Some bytes. Two parameters that have to agree, and nothing makes them agree.

Row five: `fprintf` with `%-16s %lu`. A formatted line, checked by nobody. GCC's `-Wformat` catches some of it, for literal format strings, for the built-in types. That's the whole safety net.

Today each one becomes a type that says it.

And each row is a task in today's exercise. Row one is task 1, `expected`. Row two is task 2, `optional`. Row five is task 3, `println`. The others are the at-home tasks.

> ASK: In the code you maintain, which of these five shapes is the most common? (Expect: bool plus out-param, or nullable pointer returns. In defense code, pointer plus length is everywhere at hardware and C boundaries.) Say: "Good, then that's the slide to pay the most attention to. Every one of these has a replacement in the next hour and a half."
>
> IF AHEAD: A useful code-review question for any signature: "what can I learn about this function without opening the .cpp?" If the honest answer involves the word "unless" or "if", that condition belongs in the type. It's the same instinct as `[[nodiscard]]` from Session 1, extended from "don't ignore this" to "here's exactly what you might get back".

Before we fix them one by one, here's the whole vocabulary on one slide.

5. The ownership vocabulary · target 0:05, ~1 min

This is the slide people screenshot. It comes back at the end as the interface checklist, filled in.

Two columns matter: "Owns?" and "Null?". **A signature should answer both of those questions without a comment.**

Walk down it. `T&`: doesn't own, never null, "I need one and it exists". `T*`: doesn't own, maybe null. That's really the C++98 spelling of "optional reference". `unique_ptr`: owns, alone.

`shared_ptr`: owns, shared, and "lifetime is now a runtime question", which is exactly why you should be suspicious of it in an interface.

(beat)

Then today's new rows. `string_view`: doesn't own, never null in the pointer sense, it's just empty. "I will only read these characters." `span`: doesn't own, "a contiguous run I do not own". `optional`: owns its value, inline, no heap, maybe empty. `expected`: owns its value inline, never empty: it always holds either the value or the error.

Notice the pattern. The two views at the top of the new rows don't own anything. The two at the bottom own everything, inline. **Views borrow; optional and expected hold.** Keep that split in your head and you'll get the lifetime questions right.

> IF BEHIND: Views borrow, `optional` and `expected` hold their value inline; a signature should answer "owns?" and "null?" by its types alone.

Let's start with the borrowers.

6. `std::string_view` · C++17 · target 0:06, ~3 min

Left side, C++11. A function that counts commas. It takes `const std::string&`, because that's what we were all taught. Look at the comments at the bottom left. Call it with a string literal, `"a,b"`, and it constructs a temporary `std::string`, which may allocate, just to count commas. Call it with a buffer and a length from a C API, and there's no overload at all. So people wrote three overloads, or they wrote one that forces everybody to make a `std::string`.

Right side, C++17. Same body, character for character. Only the parameter type changed: `std::string_view`, by value.

A `string_view` is a pointer and a length. Sixteen bytes, trivially copyable, pass it by value.

That one signature now accepts a literal, a `std::string`, a substring of something else, or a pointer-and-length from a C buffer. No allocation in any of those cases.

(pause)

Why by value? Because it's two words. On x86-64 and on AArch64 it goes in two registers. Passing it by `const&` means passing a pointer to a pointer and a length, which is strictly worse.

This is `split` in the exercise, at-home task 5: it returns a vector of views into the caller's line buffer, so the parser doesn't allocate a string per field anymore.

> DO: Right pane: `demos/s02/string_view_basics.cpp`. Terminal: `./build/demos/s02/demo_s02_string_view_basics`. Expect two lines: `2 2 2` and `1725, 1 1 2`. Point at line 42: the same `s` goes to both functions, and the literal `"a,b,c"` goes to the new one with no temporary string. Recovery: if the binary is missing, `cmake --build build --target demo_s02_string_view_basics`, or just walk the code; the output is not the point.

IF AHEAD: The implicit conversion goes from `std::string` to `string_view`, via `std::string`'s conversion operator, and not the other way. That asymmetry is deliberate: going from view to string is an allocation, so it has to be explicit, `std::string(sv)`. You'll see that rule again on slide 9. Also: `string_view` is `basic_string_view<char>`; there are `wstring_view`, `u8string_view`, and friends, same deal.

IF BEHIND: `string_view` is a pointer and a length, passed by value, and one signature now takes literals, strings, and slices with no allocation.

Everything you can call on one, on a single slide.

7. `string_view` at a glance · C++17 · target 0:09, ~1 min

One minute, a reference slide: everything you will call on a view, one line each, with the trap on the right. **Every row is constant time except the searches, and none of them copies a character.**

Two rows to point at. `data()`: there is no NUL terminator, so it does not go to `printf("%s")` or to any C API that expects one; that is the appendix slide on "not a C string", and lab 2.1 touches it. And `remove_prefix`: no bounds check, so a size check belongs right above it.

The last row is the exit: `std::string(sv)` when you need to keep the characters. Explicit on purpose, so a copy is visible in the code.

Screenshot it; the cheat sheet handout has the same table. Now the price of all this.

8. The dangling rule · C++17 · target 0:10, ~3 min

Slow down here. This slide is the entire cost of every view type we'll see today.

A view is a reference. It is valid exactly as long as the thing it views.

(pause)

There are four ways to break that rule, and they're numbered in the code.

Number one, line 22 on the right: `std::string_view a = make();`. `make()` returns a `std::string` by value. That string is a temporary. It dies at the semicolon. `a` now points at freed memory, and the next line that reads `a` is undefined behavior. It will usually "work" in a debug build, which is what makes it vicious.

Number two, line 18: returning a view from a function where the characters belong to that function. Here it's `s.substr(0, 5)` on a local `std::string`. One detail worth knowing: `std::string::substr` returns a brand-new `std::string`, so the view is actually of a temporary made from the local. Either way, everything it could point to is gone when the function returns.

Number three, line 12: a `string_view` stored as a struct member. That's not automatically a bug. But now `Config` is only safe while whoever owns those characters outlives it, and nothing in the type says who that is.

Number four, line 23: `std::string("x").c_str()`. This is the C++98 edition of the same bug. We've had this one for twenty-five years. A `const char*` has always been a view; we just didn't call it that.

(pause)

Now the good news. Three of these, Clang 18 catches under `-Werror`: `-Wreturn-stack-address` for the return and `-Wdangling-gsl` for the two temporaries. That's why they're behind `SHOW_ERRORS`: the repo builds with `-Werror`, so with that block enabled it doesn't build.

> DO: Switch to Compiler Explorer tab 1: `string_view_dangling.cpp`, Clang 18, flags `-std=c++23 -Wall -Wextra -Werror -DSHOW_ERRORS`. Expect three errors: "returning address of local temporary object" on line 18, and "object backing the pointer will be destroyed at the end of the full-expression" on lines 22 and 23. Point at each and match it to its number. Then delete `-DSHOW_ERRORS` and show it builds and prints `rpm,3`. Recovery: if Compiler Explorer is slow, run locally: `clang++ -std=c++23 -Wall -Wextra -Werror -DSHOW_ERRORS -fsyntax-only demos/s02/string_view_dangling.cpp`.

(beat)

Notice what the compiler can't catch: number three, the stored member. That one is on you, and on code review.

Now look at the safe pattern at the bottom, `sensor_of`, line 30. A view comes in, and a view of the **same buffer** goes out. The caller owns the buffer, the caller holds the result, and the lifetime is the caller's problem, which is where it belongs.

So here's the rule I want you to use in review:

A `string_view` parameter is free. A `string_view` member or return value needs a lifetime argument written down.

> ASK: Which of the four would you expect to survive into production most often? (Expect: the struct member, number three, because no compiler warns and it works until someone changes who owns the string. Some will say number one.) Say: "Right. Members are the ones to grep for. In review, any `string_view` member should come with a comment saying who owns the characters, or it should be a `std::string`."
>
> IF AHEAD: Two practical tools. First, AddressSanitizer: the comment on line 40 says it, run `dangling()` under `-fsanitize=address` and numbers one and four fail loudly with heap-use-after-free. If your project has a sanitizer CI job, these get caught in testing even when the compiler misses them. Second, a subtle one in `sensor_of` itself: if the line has no comma, `find` returns `npos`, and `npos + 1` wraps to zero, so `substr(0)` returns the whole line. That's well-defined, and here it's even arguably fine, but `npos + 1` is a pattern worth stopping on in review every time you see it.

The second half of the price, the missing NUL terminator, is in the appendix: a view is not a C string, so `%.*s`, not `%s`. Lab 2.1 will remind you. Now, when not to use one.

9. When to still take `const std::string&` · C++17 · target 0:13, ~1 min

Three rules, and the third one is the one people miss.

Take `std::string_view` when you only read. Parsers, lookups, comparisons, logging.

Take `const std::string&` when you're going to call `c_str()`, or pass it on to something that wants a `std::string`. If you take a view there, you force a copy that the caller might have already had for free. Copy-then-copy.

(beat)

And the third: **take `std::string` by value when you're going to keep it.** A constructor, a setter: you take it by value and `std::move` it into the member. The caller decides. If the caller has a temporary or moves in, that's zero copies. If the caller passes an lvalue they want to keep, that's exactly one copy, made at the call site. You can't do better than that with `const&`, which always copies once inside.

And never `const std::string_view&`. It's already two words. Pass by value.

(pause)

> ASK: You have `void set_name(std::string_view n) { name_ = n; }`. What's wrong? (Expect: `name_ = n` doesn't compile, because there's no implicit conversion from view to string; you'd need `name_ = std::string(n)`, which always copies, even when the caller had a temporary string it could have moved.) Say: "Exactly. For a sink, by-value-then-move beats the view."
>
> IF AHEAD: Clang-tidy has `modernize-pass-by-value`, which finds constructors that take `const T&` and copy into a member, and rewrites them to by-value-then-move. Also `performance-unnecessary-value-param` for the opposite mistake. Both are low-noise and worth turning on in a CI job.
>
> IF BEHIND: View to read, `const std::string&` when you need `c_str()`, by value then `std::move` when you keep it, and never a view by reference.

`string_view` was for characters. Now the same idea for everything else.

10. `std::span` · C++20 · target 0:14, ~2 min

Left side: `mean_cpp11(const std::vector<double>&)`. **It says "a vector" when it means "some doubles".**

Look at the comments under it. Pass a C array: doesn't compile, a C array isn't a vector. Pass a sub-range: `{v.begin()+1, v.end()}` builds a new vector, which copies. So the signature forced a container choice onto every caller.

Right side, C++20. `mean(std::span<const double> v)`. Same body. Now look at the calls at the bottom.

Line 34 on the right: a vector. Line 35: a C array, and the size is deduced from the array type, nobody passes a length. Line 36: a `std::array`. Line 37: a slice, `std::span{v}.subspan(1,`

2), which is elements two and three, no copy. Line 38: {v.data() + 2, 2}, a pointer and a count. That's the C interface, and it still works, but now the pointer and the count travel together as one value.

(pause)

One signature, every contiguous container, no copy, no ownership.

span<const double> is the read-only one. If the function should write into the caller's buffer, take span<double>. That's the next slide.

In the exercise this is at-home task 6: value_range, compute_stats, and top_n_by_value take std::span<const Record>, and the tests pass a C array and a sub-span to prove it.

> DO: Terminal: ./build/demos/s02/demo_s02_span_basics. Expect 2.50 20.00 6.00 2.50. Point at the last number: the slice {2, 3} averages to 2.5. Recovery: none needed; read the expected output aloud.
>
> IF AHEAD: One honest gap: mean divides by v.size(), so an empty span divides by zero. In floating point that's a NaN, not a crash, which is arguably worse: it propagates silently. The vector version had the same bug. The exercise fixes exactly this for value_range by returning an optional, task 7. Keep that in mind on the optional slides: an empty input is a "maybe" result.
>
> IF BEHIND: span<const T> is "some T's I don't own"; it takes vectors, arrays, C arrays, slices, and pointer-plus-count with no copy.

The whole API on one slide, then the embedded use case.

11. span at a glance • C++20 • target 0:16, ~1 min

One minute, the reference slide for span. Construction from anything contiguous, the const T versus T distinction (take const unless you write through it), the static extent when the size is part of the type.

Everything on this slide is constant time, and nothing on it is bounds-checked. subspan takes an offset and a count, not an end index; lab 2.4 will remind you. as_bytes is the row that matters at a hardware boundary, which is the next slide.

12. span at a hardware boundary • C++20 • target 0:17, ~2 min

Buffers, register windows, DMA descriptors: a pointer and a length, made into a type.

Look at checksum on the right, line 11. It takes std::span<const std::byte>. That parameter says: "any object's bytes, any buffer, I won't write, I won't keep it."

Now the three calls in demo. Line 20: std::as_bytes(std::span{text}). The span is deduced as span<const char> from the string view, then as_bytes reinterprets it as bytes. A string's bytes.

Line 21: std::as_bytes(std::span{&p, 1}). A span of one Packet, reinterpreted as eight bytes. A struct's bytes, which is exactly what you want for a checksum over a wire struct.

Line 22: `checksum(fixed)`. `fixed` is `std::span<std::byte, 8>`, static extent. Exactly eight, enforced when the caller builds it. And it converts to the dynamic span `checksum` wants.

(pause)

Now `std::byte`, which is C++17. **`std::byte` is raw memory, not a number.** It's an `enum class` over `unsigned char`. So `b + 1` doesn't compile. You get bit operations, shifts, and `std::to_integer` when you really want the number, which is what line 13 does. That's the feature: you can't accidentally do arithmetic on what's supposed to be opaque memory.

In the exercise, `crc16` becomes `crc16(std::span<const std::byte>)` with a `string_view` overload that calls `as_bytes`. At-home task 6.

> DO: Terminal: `./build/demos/s02/demo_s02_span_bytes`. Expect 38. Point at line 30: `std::span{"abc"}` is a span over the string literal's array, which is four characters including the NUL, so the checksum is 97 plus 98 plus 99 plus 0, mod 256, which is 38. Recovery: if asked why not 294, it's the `uint8_t` wraparound on line 13.
>
> ASK: Why use `std::byte` here instead of `std::uint8_t`? (Expect: intent and type safety; `uint8_t` is a number and arithmetic on it compiles silently. Some will add aliasing: both `unsigned char` and `std::byte` may alias any object, `uint8_t` usually happens to be `unsigned char` but isn't guaranteed to be.) Say: "Both right. The type says 'these are bytes, not values', and the compiler holds you to it."
>
> IF AHEAD: A real caution for this audience: `as_bytes` over a struct includes its padding bytes, and their values are unspecified. `Packet` here has no padding, two plus two plus four, but add a `uint8_t` field and your checksum now covers garbage. Either `static_assert(std::has_unique_object_representations_v<Packet>)`, which is C++17 and fails if there's padding, or serialize field by field. Also, `as_bytes` uses `reinterpret_cast` internally, so it's not `constexpr`, which matters in Session 3 when we build the CRC table at compile time.
>
> IF BEHIND: `span<const std::byte>` is "some bytes", `as_bytes` gets you there from any span, and `std::byte` refuses arithmetic on purpose.

`mdspan` and transparent comparators are in the appendix if you want them. The takeaway.

13. Views takeaway · target 0:19, ~1 min

`string_view` and `span` are the same idea: a non-owning view of contiguous memory, passed by value.

Their whole cost is one rule: a view lives no longer than what it views.

(pause)

As parameters, that rule is free. The caller's buffer outlives the call. As members or return values, write the lifetime down, or use an owning type.

Monday morning: find one `const std::string&` parameter that only reads, and change it to `std::string_view`. One. Check that every caller still compiles. That's it.

Time check: 0:20, and lab 2.1 starts now. Block 2 has the most new types, so keep moving when you come back.

After the lab: the types that hold their value.

14. Lab 2.1: views over data · target 0:20, ~10 min

Ten minutes. Three TODOs, in order. Build and run after each one. The link is on the slide; the same file is `labs/s02/lab1_views.cpp` if you built the repo.

> DO: Put the lab slide up and paste the starter link into chat. Say nothing for the first minute while people get the file open. Then: "The output tells you whether each step worked. The label line prints garbage until TODO 3 is done; that is deliberate."

What to watch for:

- TODO 1: people change the parameters and then also "fix" the calls in `main`. Nothing in `main` needs to change; that is the point of `string_view`. If someone's build breaks here, they changed `text.compare(...)`: it still exists on `string_view`.
- TODO 2: `checksum(raw + 2, 3)` is the one that needs thought. `std::span(raw).subspan(2, 3)`, or `std::span(raw + 2, 3)`. Both are right; the first says what it means.
- TODO 3: the fix is `std::string`, not `const std::string&` and not `std::string_view` with a `static`. If someone says "but it printed fine for me", that is the lesson: undefined behavior is not required to misbehave.

At seven minutes, paste the solution link. Walk the three diffs in two minutes, slowest on TODO 3.

> ASK: "Which of the three would your code review have caught?" Expect: one and two, because they are visible in the signature; three only if someone knows the rule. Then: "That is why the rule is on its own slide."

At ten minutes, move on whether or not people are done. Nothing in block 2 depends on this lab.

15. `std::optional` · C++17 · target 0:30, ~2 min

Left side, C++11: `find_cpp11` returns `const SensorConfig*`. `nullptr` means not found.

A nullable pointer says two things: "maybe", and "the object lives somewhere else". The second thing is usually incidental, and it drags a lifetime question in with it. How long is that pointer good for? Until the table is modified? Forever? The signature can't say.

Right side, C++17: `std::optional<SensorConfig>`. **`optional` says only "maybe", and it holds the object inline.** No pointer, no heap, no lifetime question. Line 22: `return s;` copies the found config into the optional. Line 23: `return std::nullopt;` for not found.

(pause)

Notice the trade: the pointer version returns an address; the optional version returns a copy. For a small config struct, that copy is nothing. For a big object you'd think twice, and we'll come back to that in the questions.

In the exercise, this is in-class task 2: `find_sensor` returns `std::optional<SensorConfig>`.

> IF AHEAD: In the demo, `table` is a `constexpr` array of structs holding `string_view`s to literals. That's why it's safe for `SensorConfig::name` to be a view: string literals live for the whole program. At-home task 9 makes exactly that point. This is the "lifetime argument written down" from slide 8: the argument is "they're literals".
>
> IF BEHIND: `optional<T>` means "maybe a T", held inline, replacing the nullable pointer and its lifetime question.

How do you use one?

16. `optional` mechanics · C++17 · target 0:32, ~2 min

A `T` plus a `bool`, with pointer-like syntax and no allocation.

Line 29 on the right: `if (auto cfg = find("rpm"))`. Contextual conversion to `bool`: it's true if there's a value. This form, declare-and-test in one `if`, is the one to prefer, because `cfg` only exists inside the branch where it's valid.

Line 30: `cfg->max`. Arrow and star work like a pointer, and like a pointer **they're unchecked: dereferencing an empty optional is undefined behavior.**

Line 32: `value_or(...)`, the default when empty.

Line 34: `c->max = 1.0`. You can mutate it, because the optional holds the object itself, not a pointer to a const table entry. Line 35: `reset()` empties it.

(beat)

Line 36, commented out: `.value()` on an empty optional throws `std::bad_optional_access`. So: star and arrow when you've checked, `value()` when you want a check that throws, `value_or` when you have a sensible default.

> DO: Terminal: `./build/demos/s02/demo_s02_optional_basics`. Expect `12000.000000` then `0.000000 0`. Point: the second line is the `value_or` default and `has_value()` after `reset`. Recovery: read it aloud.

Size: `sizeof(std::optional<double>)` is sixteen on our platforms. The `bool` plus padding to eight-byte alignment. And there's no `optional<T&>` in C++23; C++26 adds it. Today, use a pointer or `std::reference_wrapper`.

> IF AHEAD: Under `-fno-exceptions`, `value()` can't throw, so libstdc++ calls `abort()` instead. That's actually a reasonable behavior for an embedded build, but it means `value()` is a crash, not an error path. If your codebase bans exceptions, the honest pattern is `if (opt)` then `*opt`, and treat `value()` like an assert. Also,

libstdc++'s `_GLIBCXX_ASSERTIONS` makes `operator*` on an empty optional trap in debug builds; libc++'s hardening modes do the same.

IF BEHIND: Test with `if (auto x = f())`, use * and `->` only after the test, `value()` throws, `value_or` gives a default.

The whole API on one slide first.

17. `optional` at a glance · C++17, C++23 · target 0:34, ~1 min

One minute, the same kind of slide for `optional`. Top half is C++17: ask with `has_value` or the `if`, access with * and `->` when you have asked, `value()` when you want the throw, `value_or` when you have a default.

`value_or` returns a `T` by value: it copies. With an `optional<string_view>` that is cheap and it is still a view, which is where lab 2.2's first TODO catches people.

Bottom half is C++23, the monadic three. `transform` for a function that cannot fail: plain result in, wrapped result out. `and_then` for a function that can: it returns the `optional` itself. `or_else` for the fallback that has to compute something. If you remember one rule: **plain result means `transform`, optional result means `and_then`.**

Now where an `optional` belongs in an interface.

18. `optional` as return, member, parameter · C++17 · target 0:35, ~1 min

This is an opinionated slide, and I'm telling you it's opinionated.

Return type: yes. "Maybe a result" is exactly what `optional` is for. It replaces `-1`, `nullptr`, empty string, and `bool` plus out-parameter.

Member: sometimes. A lazily computed value, a configuration field that's genuinely allowed to be absent. Not as a substitute for a proper default value.

Parameter: rarely. `f(std::optional<int> timeout)` usually reads better as two overloads, or a default argument. The exception is when you're forwarding a maybe-value through a layer that doesn't care.

(pause)

And never `std::optional<bool>`. Three states, two names. `if (flag)` tests "is there a value", not "is it true". That's a bug factory. **If you need three states, use an enum with three names.**

ASK: What does `if (opt_bool)` test when `opt_bool` holds `false`? (Expect: it's true, because the optional is engaged, even though the value is false.) Say: "And that's the bug. Somebody will write that line."

IF AHEAD: The same trap applies to `optional<T*>` and `optional<int>` where 0 is meaningful: two levels of "nothing". If you see `std::optional<std::unique_ptr<T>>` in review, ask what the difference

between "no optional" and "null pointer" is supposed to be. Usually there isn't one, and the plain `unique_ptr` was enough.

IF BEHIND: Return types yes, members sometimes, parameters rarely, `optional<bool>` never.

Now the C++23 part, which removes the nested ifs.

19. Monadic `optional` · C++23 · target 0:36, ~3 min

Left side, C++17: `units_cpp17`. Look up the sensor; if it's not there, return empty; otherwise return its units. Three lines, one `if`. Fine. Now imagine four lookups deep. That's the shape that grows into an arrow of nested ifs.

Right side, C++23. `units`, line 28. Read it top to bottom: `find_sensor(name)`, then `.transform(...)` with a lambda that takes the config and returns its units, then `.value_or("")`.

`transform` applies a function that can't fail, and keeps the result wrapped. `optional<SensorConfig>` goes in, `optional<string_view>` comes out. If the input was empty, the lambda never runs and you get an empty optional out.

(pause)

Now `doubled`, line 34. Three operations, one for each method.

`and_then`, line 36: the lambda returns an `optional` itself. **`and_then` is for a step that can fail.** Here, anything over 100 is rejected with `nullopt`. `transform` would give you an optional of an optional; `and_then` flattens it.

`transform`, line 37: doubles it. Can't fail.

`or_else`, line 38: runs only if everything before produced empty. Recovery: return zero.

So: `and_then` may fail, `transform` can't fail, `or_else` recovers. That's the whole vocabulary, and it's the same three words on `expected` in twenty minutes.

In the exercise, at-home task 8: the report's units lookup becomes exactly the `transform` and `value_or` line on the right.

DO: Terminal: `./build/demos/s02/demo_s02_optional_monadic`. Expect `rpm rpm 6`. Point: `to_int("abc")` in this demo returns the length, 3, so `doubled` gives 6. It's a stub, so don't read anything into it. Recovery: read it aloud.

IF AHEAD: Note the explicit return type on the `and_then` lambda, `-> std::optional<int>`. Without it, the conditional `n > 100 ? std::nullopt : std::optional{n}` still works because `nullopt_t` converts to `optional<int>`, but in general, write the return type on `and_then` lambdas: if two `return` statements deduce different types, the error is long and lands inside the library. Also, `value_or("")` on line 31 returns a `string_view`, so the `std::string(...)` around the whole chain is the one copy, at the end.

> IF BEHIND: `and_then` for steps that can fail, `transform` for steps that can't, `or_else` to recover; no nested ifs.

There's one trap in `transform`, the pointer-to-member one, that the at-home exercise makes you hit on purpose; it's in the appendix. Now the "which one" type.

20. `std::variant` · C++17 · target 0:39, ~2 min

Left side, C++11: the tagged union. An enum saying which member is live, a union of `int` and `double`, and a `std::string` sitting beside the union because it can't go inside: non-trivial types in a union needed hand-written constructors and destructors.

The compiler enforces nothing. Set `kind` to `Text` and read `d`, it compiles. Forget to update `kind` on assignment, it compiles.

Right side, C++17: `using Value = std::variant<int, double, std::string>;`. **Exactly one of these is alive, and the variant knows which.**

(pause)

Line 22 on the right: `holds_alternative<int>` asks "is it an int?", then `get<int>` gets it. Line 23: `get_if<double>(&v)` returns a pointer, `nullptr` if it isn't a double. That's the check-and-get-in-one form, and it's the one I'd use. Line 24: `index()` is the position, 0, 1, or 2.

Line 25, commented: `get<int>` on a variant holding a string throws `std::bad_variant_access`.

And the string just works. Variant runs the right constructor and destructor for whichever alternative is alive.

> IF AHEAD: Construction picks the alternative by overload resolution, which has had surprises. Before a C++20 fix (P0608), `std::variant<std::string, bool> v = "abc";` chose `bool`, because pointer-to-bool is a standard conversion and beats the user-defined conversion to `std::string`. Current GCC and Clang choose `std::string`, but if you're on an older certified toolchain, check. In general, construct a variant with `std::in_place_type<T>` when there's any doubt.
>
> IF BEHIND: A variant is a type-safe tagged union: `get_if` to check and get, `index()` for which one, `get` throws on the wrong type.

First, the whole API on one slide.

21. `variant` at a glance · C++17 · target 0:41, ~1 min

One minute, the reference slide for `variant`. Construction: default is the first alternative, which is why `std::monostate` goes first when "nothing yet" is a real state. Assigning an alternative switches to it.

Then two ways to look inside. **Ask, then get: `holds_alternative` and `get`, or `get_if`, which does both and hands you a pointer or null.** That is for one alternative in one place. And `visit`, which hands whatever is inside to a callable that must accept every alternative.

(beat)

That word, every, is the difference between the two, and the next slide is about it: a chain of `holds_alternative` checks compiles unchanged when you add a fourth message type next year. `visit` does not.

22. `std::visit` and the overload-set visitor · C++17 · target 0:42, ~3 min

Slow down here. Two lines of template code that appear in every codebase that uses variant.

Line 30 on the right. Read it piece by piece.

`template <class... Fs>`: a pack of types, which are going to be lambda types.

`struct overloaded : Fs...`: the struct inherits from **all** of them. Every lambda is a class with an `operator()`, so now `overloaded` has all those base classes.

`using Fs::operator()...;`: that's a pack expansion on a using-declaration, C++17. It pulls every base's `operator()` into one overload set. Without it, the names from different bases would be ambiguous.

(pause)

Result: one object whose `operator()` is an overload set of all your lambdas.

Now `describe`, line 32. `std::visit` takes that object and the variant. It looks at which alternative is alive and calls the matching overload. The int lambda for an int, the double one for a double, the string one for a string.

How does `overloaded{ lambda, lambda, lambda }` know its template arguments? Class template argument deduction, Session 1. One caveat: in strict C++17 you need a one-line deduction guide, `template <class... Fs> overloaded(Fs...) -> overloaded<Fs...>;`. From C++20, aggregate CTAD makes it unnecessary, which is why the demo doesn't have it. The slide badge says C++17, so if your toolchain is in C++17 mode, add that line.

(pause)

Now the payoff, line 39. Add a fourth alternative to `Value`, and **`visit` stops compiling until you handle it.** That's the switch the compiler can verify.

One honest qualifier: that's only true if the new type doesn't implicitly convert to a parameter you already handle. Add `float`, and it silently goes to the `double` lambda. Add `const char*`, and it goes to the string one. For exhaustiveness you can trust, keep the parameter types exact and don't add a generic `auto` catch-all.

> DO: Terminal: `./build/demos/s02/demo_s02_variant_visit`. Expect `double 2.500000`, `index 1`, and `int 1 double 2.500000 text x`. Then live edit: add `long` to `Value` on line 19 and rebuild: `cmake --build build --target demo_s02_variant_visit`. Expect an error: a `long` argument is ambiguous between the int and double lambdas. Undo. Recovery: if the error is too long to read on screen, say "the compiler refuses, that's the point" and undo.

ASK: Why does `visit` fail for `long` but accept `float`? (Expect: `long` to `int` and `long` to `double` are both conversions, equally ranked, so ambiguous; `float` to `double` is a promotion, which wins.) Say: "Right. So the exhaustiveness guarantee comes from overload resolution, with all its rules. Exact types keep it honest."

IF AHEAD: A clean alternative some teams prefer: one generic lambda with `if constexpr` chains on `std::is_same_v<std::decay_t<decltype(x)>, T>`, ending in `else static_assert(always_false<T>)`. That gives a hard error for any unhandled type, conversions included. It's uglier, but it's the most rigorous. On performance: GCC and Clang compile `visit` for a small variant to a jump table or a switch; it is not a chain of virtual calls.

IF BEHIND: `overloaded` inherits all the lambdas, `visit` calls the matching one, and adding an alternative breaks the build until you handle it.

Now a real use.

23. Variant in practice: a message set • C++17 • target 0:45, ~2 min

A protocol with a fixed set of message types is a variant, not a class hierarchy.

Lines 11 to 13 on the right: three plain structs. `Reading` has a sensor ID and a value. `Heartbeat` has uptime. `Fault` has a code and a detail string. No base class, no virtual destructor.

Line 15: `using Message = std::variant<Reading, Heartbeat, Fault>;`. The whole protocol, one line. Anybody reading that line knows every message the link can carry.

Line 19: `handle` visits with three lambdas, one per message type. Add a fourth message to the protocol, and every `handle` in the codebase fails to compile until somebody decides what to do with it. **In a protocol, that's exactly the guarantee you want.**

(pause)

Now the bottom block, the decision rule. Variant: closed set of types, open set of operations. You can write a new visitor anywhere, without touching the types. Virtual: open set of types, closed set of operations. You can add a new derived class anywhere, but adding an operation means touching the base.

Protocols, ASTs, state-machine states: closed set of types. Variant. Plugin interfaces, where someone else adds types you've never seen: open set. Virtual.

DO: Terminal: `./build/demos/s02/demo_s02_variant_messages`. Expect three lines, `reading 3 = 41.25`, `alive 120s`, `FAULT 7: overtemp`, then the size line: `48 bytes per message` on GCC 14, smaller on Clang with libc++ because libc++'s `std::string` is 24 bytes, not 32. Point: the vector holds messages by value, contiguous, no heap except the fault's string. Recovery: read it aloud.

Size is the largest alternative, plus a small index, rounded up for alignment. No vtable pointer, no heap allocation per message, value semantics: copy it, compare it, put it in a vector.

ASK: You have a hardware abstraction layer with a base class `Sensor` and twelve derived sensor drivers, and other teams add drivers. Variant or virtual? (Expect: virtual. It's an open set of types; other teams add drivers.) Say: "Right. And the messages those drivers produce? That's a closed protocol, so variant. Both in the same system is normal."

IF AHEAD: For a message variant that crosses a wire, don't send the variant's bytes. The layout, including the index type and its position, is implementation-defined. Serialize with `index()` as a tag and the alternative's fields, then reconstruct with `std::in_place_index`. And a `std::string` inside a message means a heap allocation per fault; in an embedded link you'd use a fixed-size `std::array<char, N>` there instead.

IF BEHIND: A closed protocol is a variant of plain structs; visit is exhaustive; virtual is for open sets of types.

The details slide (`monostate`, `valueless_by_exception`) is in the appendix. One more type, briefly.

24. `std::any` · C++17 · target 0:47, ~1 min

Sometimes the set of types is genuinely open, and only the caller knows what it put in. That is the one job `std::any` has, and this slide is its whole API.

Assign anything into it; the type is erased, and all that survives is the `typeid`. Reading it back is a cast: `any_cast<int>(a)` by value throws `bad_any_cast` if it is not an `int`, `any_cast<int>(&a)` with a pointer returns `nullptr`. The same two forms as `get` and `get_if` on a variant, and no conversions at all: put in `3` and `any_cast<long>` fails.

(pause)

It heap-allocates for anything bigger than its small internal buffer, a pointer or a few pointers depending on the library. The last row is the only honest use case: a property bag at a plugin boundary or a scripting bridge.

If you can name the alternatives, use `variant`. Every read of an `any` is a cast that can fail at run time, which is exactly what `variant` and `visit` move to compile time. `any` is the type people reach for and regret; it is the right tool about once per codebase.

So how do you choose?

25. Choosing between them · target 0:48, ~1 min

Same idea as the ownership table, from the other direction: start from what you need.

Maybe one `T`: `optional`, not a pointer, not a sentinel, not `pair<T, bool>`. Exactly one of a **fixed** set: `variant`. One of an **open** set with fixed operations: a virtual base class. Anything, checked at runtime: `any`, not `void*`.

Someone else's `T`, maybe absent: still a raw pointer, non-owning. That's not a smell. `optional<T&>` doesn't exist yet.

(pause)

And the last row: a T, or the reason there's no T. That's `expected`, and that's block 3, after the lab.

> IF BEHIND: Skip the walk-through: "Start from the need: maybe, one of a fixed set, open set, anything, borrowed, or value-or-reason."

Where these came from, Boost and a decade of proposals, is in the appendix. The takeaway.

26. Maybe, either, anything: takeaway · target 0:49, ~1 min

`optional` replaces every sentinel value and every `bool` plus out-parameter.

`variant` replaces every tag-plus-union, and most small class hierarchies over a fixed set.

`any` replaces `void*`, and almost nothing else.

(pause)

Monday morning: find one function that returns `-1` or `nullptr` for "not found", and give it an `optional` return type.

Time check: 0:50, lab 2.2 now. `optional` told you "no". After the lab, the type that tells you why.

27. Lab 2.2: maybe, either · target 0:50, ~10 min

Ten minutes, three TODOs. Part A is `optional`, part B is `variant`. Do them in order; part B is the long one.

> DO: Lab slide up, starter link in chat. One quiet minute.

What to watch for:

- TODO 1: `find_unit("rpm").value_or("?")` is a `string_view`; printing it with `%s` is the same bug as the `data()` row on slide 7 (and appendix slide 62). `std::string(...)`, or `%.*s`. Half the room will hit this and it is a good thing: it is the dangling-rule's cousin.
- TODO 2: one line. If someone reaches for `and_then`, ask what the lambda returns. `transform` wraps the plain `double` for you; `and_then` expects the lambda to return the `optional` itself.
- TODO 3: the `push_back` calls in `main` change too: `Reading{3, 11.5}` instead of the five-field brace soup. People who keep the old `log.push_back({...})` lines get a long error about no matching constructor. Point at the variant's converting constructor: push the alternative, not the variant.
- Anyone done early: delete the `Fault` lambda from the `overloaded` and read the error. Exhaustive by construction.

At seven minutes: solution link, two-minute walk-through. Spend most of it on the `overloaded` block: three lambdas, one `visit`, no enum, no switch.

> ASK: "What happens when you add a fourth message kind next month and forget `handle()`?" Expect: "it doesn't compile." Then: "With the enum-and-switch version it compiled and dropped the message on the floor."

28. The error-handling landscape · target 1:00, ~1.5 min

Six ways to report failure in C++, five columns.

Exceptions: carry a reason, can't be ignored, compose by propagating, near-zero cost on the happy path, expensive when thrown. Error codes and `errno`: carry a reason, **trivially ignored**, don't compose. `bool` plus out-params: same problem, and the reason is in another parameter. `std::error_code` from C++11: typed, but still ignorable, still doesn't compose.

`optional`: composes with the monadic operations. But look at the first column: **it carries no reason.** It says "no" without saying why.

`expected`: carries a reason, composes, and the happy-path cost is one compare.

(pause)

One correction to the "Ignorable?" column for `optional` and `expected`. The types aren't `[[nodiscard]]` in the standard. If you call a function that returns an `expected` and drop the result on the floor, that compiles without a word. What the type does prevent is using the value without going through the check. To make dropping it a warning, you mark the function `[[nodiscard]]`, and slide 33 says exactly that.

`expected` is `optional` with a reason.

> ASK: In your codebase, which of these rows is the dominant one? (Expect: error codes and `bool` plus out-param, especially in embedded code with exceptions disabled.) Say: "Then you're the people this next slide is for."
>
> IF AHEAD: `std::error_code` deserves more credit than it gets: it's how `std::filesystem` and `<system_error>` report OS errors, and it pairs with `std::error_category` for custom domains. It's fine as the `E` in `expected<T, std::error_code>` if you're already in that world.
>
> IF BEHIND: Every mechanism either loses the reason, can be ignored, or doesn't compose; `expected` carries the reason and composes.

Here's the payoff.

29. `std::expected` · C++23 · target 1:01, ~2 min

Left side, C++11: `parse_cpp11`. Returns `bool`, writes the result through one pointer, writes the error through another. **The caller can ignore all three.** And notice: if the parse fails, what's in `*out`? Line 15 on the right of the file: `from_chars` wrote into it, maybe. Nobody knows. The caller has to remember not to look.

Right side, C++23. `std::expected<int, ParseError> parse(std::string_view s)`. **The value, or the reason. One return type.**

(pause)

Each failure path, lines 25, 28, 29: `return std::unexpected(ParseError::...)`. The success path, line 30: just `return v;`. No wrapper on success.

The caller can't use the value without checking, because there's nothing to use until they do. There's no half-written out-parameter lying around.

And one deletion I love: `ParseError::None`. The old enum needed a "not an error" value, because the error out-parameter had to hold something on success. With `expected`, the error side only exists when there's an error. So the enum gets honest.

This is `parse_record` from the exercise, in-class task 1.

> IF AHEAD: The comment on line 23 says "`[[nodiscard]]` by nature". Strictly, it isn't: `parse("42");` alone compiles silently. What's true is that you can't extract the value without engaging with the check. Put `[[nodiscard]]` on the function as well, and you get both. In the exercise solution, every `expected`-returning function has it.
>
> IF BEHIND: `expected<T, E>` returns the value or `std::unexpected(reason)`; no out-parameters, and no `None` error.

Mechanics, quickly, because they mirror `optional`.

30. `expected` mechanics · C++23 · target 1:03, ~1 min

Same shape as `optional`, plus an `error()` side.

Line 36 on the right: `if (auto n = parse("42"))`, then `*n`. Bool test, star, arrow, exactly like `optional`. Line 38: on failure, `e.error()` gives you the reason. Line 39: `value_or(-1)`. Line 40: `error_or`, C++23, the mirror image: the error, or a default if there wasn't one.

Line 41, commented: `.value()` on an error throws `std::bad_expected_access<ParseError>`, and that exception carries the error value, so a catch site can see why.

(pause)

Why the `std::unexpected` wrapper at all? Because `T` and `E` can be the same type. `expected<int, int>`: does `return 5;` mean the value 5 or error code 5? **`return v;` is always success; `return std::unexpected(e);` is always failure.** No ambiguity, even then.

> DO: Terminal: `./build/demos/s02/demo_s02_expected_basics`. Expect 42, then `error 2`, then `-1 0`. Point: `"4x"` parses the 4, then finds a trailing `x`, so it's `Trailing`, which is index 2. And `error_or` on a success returns the default, `Empty`, which is 0. Recovery: if Clang fails to find `<expected>`, it's building against libstdc++; the repo CMake uses libc++ for Clang for exactly this reason.

> IF AHEAD: `operator*` on an `expected` holding an error is undefined behavior, exactly like `optional`. And `error()` on an `expected` holding a value is also undefined. Both are trapped by libstdc++ assertions and libc++ hardening. Under `-fno-exceptions`, `value()` aborts.
>
> IF BEHIND: Same API as `optional`, plus `error()` and `error_or`; `unexpected` keeps success and failure unambiguous.

The whole API on one slide first.

31. `expected` at a glance · C++23 · target 1:04, ~1 min

One minute. Hold this slide next to the `optional` one in your head: **same shape, plus the error side.** `unexpected` builds the failure; a plain `return value;` builds the success. `error()` reads the reason; `value_or` and `error_or` are the two fallbacks.

Then the monadic rows, same rule as before: `transform` for a step that cannot fail, `and_then` for one that can. `transform_error` is new: it maps one error type to another, which is what you do at a layer boundary. And `expected<void, E>` for "did it work". Lab 2.3 uses the top half and one `and_then`.

Now what composition looks like in a pipeline.

32. Monadic `expected`: a pipeline · C++23 · target 1:05, ~3 min

Slow down. The type changes at every step, so I'm going to say it out loud at each line.

Three stages that can each fail, without three ifs.

Look at `scaled`, line 26 on the right. Its job: take a CSV line, read the first field, parse it as an int, check the range, scale it.

Line 27: `read_field(line)`. That returns `expected<string_view, Error>`. A field, or an error.

Line 28: `.and_then(to_int)`. `to_int` takes a `string_view` and returns `expected<int, Error>`. **`and_then` is for a stage that can fail.** So now we have `expected<int, Error>`.

(pause)

Line 29: `.and_then(check_range)`. Int in, `expected<int, Error>` out. Still `expected<int, Error>`.

Line 30: `.transform(...)`. Multiply by one half. That can't fail, so it's `transform`, and now it's `expected<double, Error>`. Which is the return type of the function.

The first failure short-circuits everything after it. If `read_field` fails, `to_int` never runs, `check_range` never runs, the lambda never runs, and the error comes out the end unchanged.

(pause)

Now `api`, line 35. This is the layer boundary. Inside, we have our parser's `Error` enum. The outside world should see an `ApiError`. `transform_error` maps the error side only: if there's a value, it passes through untouched.

`and_then` may fail, `transform` can't fail, `transform_error` changes the error type at a boundary, `or_else` recovers.

Now scroll down to line 43, the commented-out C++11 version. Three out-parameters, four ifs, and every `return false` is a place someone forgets to set `*err`.

One constraint: every stage in an `and_then` chain must use the **same** error type. You can change `T` freely; you change `E` only with `transform_error`.

> DO: Terminal: `./build/demos/s02/demo_s02_expected_pipeline`. Expect `21.000000 0 0`. Walk it: `"42,x"` reads field 42, parses, passes the range check, halves to 21. `scaled("")` fails in `read_field`, so `has_value()` is 0. `api("7")` fails in `to_int` (the stub only accepts 42), which is `BadNumber`, which maps to `ApiError::Parse`, which is 0. Recovery: read it aloud.
>
> ASK: If `to_int` returned `expected<int, std::string>` instead of `expected<int, Error>`, what happens on line 28? (Expect: compile error; `and_then` requires the callback's error type to be the same as the current one.) Say: "Right, and that's a feature. Error types change only where you say so, with `transform_error`."
>
> IF AHEAD: `and_then(to_int)` passes a function name directly. That works because `to_int` isn't overloaded. If it ever gets an overload, or if it's a function template, that line breaks with a confusing error, and you'll need a lambda. In review, prefer lambdas for anything that might grow overloads; use bare names only for stable, single functions.
>
> IF BEHIND: `and_then` chains stages that can fail, `transform` the ones that can't, `transform_error` maps error types between layers, and the first failure skips the rest.

What should `E` be?

33. Designing the error type · C++23 · target 1:08, ~1 min

Four options.

An `enum class`: the exercise's `ParseError`. Cheap, comparable, you can `switch` on it, and with `-Wswitch` the compiler tells you when you missed a case. **Right for one subsystem.**

A struct with context: kind, line number, a detail string. When the caller needs to report where and why. But keep it small. `expected` is the larger of `T` and `E`, plus a flag, plus padding. Put a `std::string` in `E`, and every return of the happy path carries 32 bytes of string you didn't use.

`std::error_code`: if you're interoperating with `<system_error>` and OS errors. Heavier to define your own category.

A `variant` of error kinds: when layers have different errors and you don't want to flatten them into one enum.

(pause)

And the rule: **mark every `expected`-returning function `[[nodiscard]]`.** The type says "check me"; the attribute enforces it.

> IF AHEAD: Sizes in the exercise: `expected<Record, ParseError>` is `sizeof(Record)` plus 8, because the `bool` flag gets padded to `Record`'s alignment. That's fine for a return value; it's in registers or the return slot. Where it matters is if you store them: a vector of a million `expected<double, BigError>` is a million copies of the largest one.
>
> IF BEHIND: Usually an `enum class`; a small struct when callers need context; always `[[nodiscard]]` on the function.

Now the debate.

34. `expected` vs exceptions · C++23 · target 1:09, ~1 min

Not a replacement. A second tool, for a different kind of failure.

Exceptions still win for constructors: they have no return value. For failures that really are exceptional: out of memory, a broken invariant. And for deep call stacks where every layer would just forward the error: exceptions do that forwarding for free.

`expected` wins for failures that are **expected**. Bad input. Not found. Timeout. Hot paths, where you don't want unwinding machinery. `-fno-exceptions` builds. And code where you want "what can fail here" to be readable in the signature.

(pause)

What it costs: a branch at every call site, a bigger return type, and the discipline not to call `.value()` blindly.

Here's my position. **In an embedded or safety-critical codebase that already bans exceptions, `expected` is what makes that policy defensible instead of a workaround.** Before, "no exceptions" meant error codes, which are ignorable and don't compose. Now it means a typed, composable result. In application code, use both, by the rule on this slide.

> ASK: Who here works in a codebase that compiles with `-fno-exceptions`, or a coding standard that bans `throw`? (Expect: a fair number of hands; AUTOSAR-style and MISRA-style rule sets restrict exceptions, and many embedded targets disable them.) Say: "Then `expected` is the most important type in this session for you. Everything else is nice; this one changes what your error handling can look like."
>
> IF AHEAD: The cost argument is subtler than it looks. With table-based exceptions, the happy path is close to free and the throw is expensive, often microseconds. With `expected`, every call pays a compare and branch, which the branch predictor almost always gets right. So for failures that happen 1 in a million, exceptions are cheaper per

call; for failures that happen 1 in 10, `expected` wins easily. And deterministic timing, which matters in real-time code, favors `expected`.

IF BEHIND: Exceptions for constructors and truly exceptional failures; `expected` for expected failures, hot paths, and no-exceptions builds.

The `expected<void, E>` case is in the appendix: same type, no value, for "did it work". The takeaway.

35. `expected` takeaway · target 1:10, ~1 min

`expected<T, E>` is `optional<T>` with a reason, and it's the piece that makes "no exceptions" a policy rather than a workaround.

Use it for failures you expect. Keep exceptions for the ones you don't.

(pause)

Monday morning: take one `bool f(..., Error* err)` function and give it an `expected` return.

Time check: 1:11. No lab yet; formatting is the second half of this block, and it's the most immediately gratifying part of the day. The lab at 1:20 covers both.

36. Why `printf` and `iostream` both lost · target 1:11, ~0.5 min

We've had two output systems for thirty years, and both lost.

`printf`: the format string and the arguments are checked by nobody. `%s` with an `int` is undefined behavior. You can't teach it your types. And you cast everything: `%lu` with a `size_t` needs a cast on some platforms and not others.

`iostream`: stateful. `std::hex` sticks until you unset it, so one function's output changes the next function's. Verbose: `setw`, `setprecision`, `fixed`. Slow, with locale and virtual dispatch per operation.

(beat)

`std::format` and `std::print`: `printf`'s ergonomics, checked at compile time, extensible to your types, and fast.

If your codebase is pre-C++20: `{fmt}` is the library `std::format` was standardized from. It's a near drop-in, and migrating from `fmt::` to `std::` later is mostly a namespace change.

> IF BEHIND: `printf` is unchecked, `iostream` is stateful and verbose; `format` fixes both.

Here's what it looks like.

37. `std::format` · C++20 · target 1:12, ~2 min

Left side, C++11: `snprintf` into a 64-byte buffer, return it as a string. A buffer, a size, and a format string checked by nobody.

Right side, C++20: `std::format("{:.3f}", v)`. Returns a `std::string`. No buffer, no size.

A mismatched format string is a compile error, because the format string is checked at compile time. Technically, `std::format` takes a `std::format_string`, whose constructor is `consteval`. So when you pass a literal, the parse runs during compilation, against the actual argument types.

(pause)

Let me show you.

> DO: Compiler Explorer tab 3: `auto s = std::format("{:d}", 3.14);` on GCC 14, `-std=c++23`. Expect a compile error: the call to the `consteval` constructor isn't a constant expression, with a note pointing at the invalid presentation type for a floating-point argument. Point: no run, no test, the build fails. Then change `{:d}` to `{:.2f}` and it compiles. Recovery: if the tab is broken, describe it and add it to the follow-up chat message.

This is `serialize(double)` in the exercise.

> IF AHEAD: The check only happens for compile-time format strings. If the format comes from a variable, you need `std::vformat`, on the format table at slide 40, and errors become a `std::format_error` exception at run time. Also, unlike `printf`, `{}` with no type always works: the type comes from the argument, not the string. That alone removes most of the `%lu` versus `%zu` versus `%llu` portability bugs.
>
> IF BEHIND: `std::format` returns a string and checks the format string against the arguments at compile time.

The syntax is the only new thing to learn.

38. The format spec mini-language · C++20 · target 1:14, ~0.5 min

Everything `printf`'s percent could do, in a syntax you can read.

The grammar: inside the braces, optional argument index, colon, then fill, align, sign, hash, zero, width, precision, type. Same order as `printf`, mostly.

Walk the comments on the right. Line 26: `{:.3f}`, precision, 3.142. Line 27: `{:<16}`, width 16, left-aligned. Line 28: `{:>8}`, right. Line 29: `{:^9}`, centered, which `printf` couldn't do. Line 30: `{:04X}`, zero-padded uppercase hex. Line 31: `{:#010x}`: the hash adds `0x`, and the width includes the prefix. Line 32: always show the sign. Line 34: positional, `{1} {0}` prints `b a`. Line 35: a fill character, stars. Line 36: double braces to print a literal brace.

(pause)

The translation table for today's exercise is four entries: `%-16s` is `{:<16}`, `%.3f` is `{:.3f}`, `%04X` is `{:04X}`, and `%lu` is just `{}`. That's in-class task 3.

> DO: Terminal: `./build/demos/s02/demo_s02_format_specs`. Expect the eleven bracketed lines matching the comments, then `3.142 3.142`. Point at the

`[left |]` line: the brackets come from the `show` helper, and the pipe proves the padding is there. Recovery: read the comments on the slide.

IF AHEAD: Two migration traps. First, default alignment: in `format`, numbers right-align and strings left-align by default. In `printf`, `%5s` right-aligns a string. So `%5s` translates to `{:>5}`, not `{:5}`. Second, hex of a negative number: `printf("%x", -1)` prints `ffffffff`, because it reinterprets as unsigned. `format("{:x}", -1)` prints `-1`. If you're dumping register values that are stored in signed types, cast to the unsigned type first.

IF BEHIND: Fill, align, width, precision, type, after the colon; the four translations for the exercise are on this slide.

Now let's print it.

39. `std::print` and `std::println` · C++23 · target 1:14, ~0.5 min

Left side: `fprintf` with `%-16s`, `.c_str()`, and a cast to `unsigned long`. Right side: `std::println(out, "{:<16} n={} mean={:.3f}", name, n, mean)`. The `.c_str()` is gone, the cast is gone, and the newline is implicit.

`std::print` is `printf` ergonomics with `format` safety. Overloads for `FILE*` and `std::ostream`. On Windows consoles, it writes Unicode correctly.

The `FILE*` overload is why the exercise's `write_report` keeps its `FILE*` parameter and the tests keep using `tmpfile()`.

(beat)

The slide says `println()` with no arguments prints a newline. That overload is C++26, not C++23. In C++23, write `std::println("")`.

DO: Terminal: `./build/demos/s02/demo_s02_print`. Expect two identical `rpm n=5 mean=4811.000` lines, then `1 and done`. Then watch the order: `to stderr` may appear **before** `no newline;` , because stdout is line-buffered and that text has no newline yet, while stderr isn't buffered. Point it out: that's ordinary C stdio, not a `print` bug. Recovery: if the order looks normal, say "on a terminal it usually swaps; in a pipe it definitely will".

IF BEHIND: `println` replaces `fprintf`, drops the casts and the `c_str()`, and adds the newline.

Before your own types, the five calls on one slide.

40. `format`, `print`, `println` at a glance · C++20, C++23 · target 1:15, ~1 min

One minute. One syntax, five destinations. **`format` builds a string, `format_to` writes through an iterator, `format_to_n` caps the length, `print` and `println` go straight to a stream.** The compile-time check on the format string applies to all of them.

`vformat` is the exception: a format string that is only known at run time, and the check cannot help you there, so errors become exceptions. Use it when the format comes from a config file, and nowhere else.

Last row: ranges and tuples format themselves in C++23, `{::.3f}` applies a spec to each element.

Now your own types.

41. `std::formatter` for your own types · C++20 · target 1:16, ~2 min

Slow down. This is at-home task 4, and it's where people get stuck alone.

`printf` could not print a `Record`. `format` can, once you tell it how. You tell it by specializing `std::formatter` for your type.

Two patterns.

Top block, the enum-like type, `Status`. Line 17 on the right: `struct std::formatter<Status> : std::formatter<std::string_view>`. **Inherit from the string-view formatter.** You get its `parse` for free, which means width, alignment, and fill all work on your type with no code. Then `format`, line 19: convert the status to a string view and hand it to the base class's `format`.

(pause)

Bottom block, the struct, `Record`. Two members to write.

`parse`, line 29: it sees the spec text. We accept only an empty spec, so we return `ctx.begin()`, which should be pointing at the closing brace. If someone writes `{:x}` for a `Record`, that's a compile error.

`format`, line 31: delegate to `std::format_to(ctx.out(), ...)`. Look at the format string: triple braces at each end. Two of them are an escaped literal brace, one opens or closes the replacement field. And the last field, `{}`, formats `r.status`, which uses the `Status` formatter from the top block. Formatters compose.

Make `format` a template on the context type. The standard allows any `basic_format_context`, and libc++ actually checks at compile time that your formatter works with one it makes up. A `format` that takes `std::format_context&` exactly compiles on GCC and fails on Clang with libc++.

> DO: Terminal: `./build/demos/s02/demo_s02_formatter_custom`. Expect `{7,"rpm",1.500,ok} [    fault] 18`. Point: the `Record` prints with its own format; `{:>8}` on `Status::Fault` right-aligns `fault` in 8 columns, which we got for free by inheriting; and 18 is the length of the formatted record. Recovery: read it aloud.
>
> ASK: Why does `{:>8}` work on `Status` but would fail on `Record`? (Expect: `Status` inherited the string-view formatter's `parse`, which understands width and alignment; `Record`'s `parse` accepts only an empty spec.) Say: "Exactly. Inheritance

gave `Status` the whole spec language. `Record` would need its own `parse`. Next slide."

IF AHEAD: Note that `format` is `const`. C++23 requires that a formatter's `format` be callable on a const formatter, and libraries check it. So any state that `parse` reads from the spec has to be stored in members by `parse` and only read by `format`. That's exactly the design on the next slide. Also, if `std::formatter<Record>` exists, `std::println("{}", records)` prints a whole vector of them with range formatting, appendix slide 71, for free.

IF BEHIND: Enum-like types inherit `formatter<string_view>`; structs write `parse` and a templated `format` that calls `format_to`.

Formatters with their own spec, formatting ranges, `format_to` into a fixed buffer, and the migration path are all in the appendix. The takeaway.

42. `std::formatter` at a glance · C++20 · target 1:18, ~1 min

One minute, the anatomy of a `formatter`. Two members: `parse` reads the spec and returns where it stopped; `format` writes the value to `ctx.out()` and returns the iterator. **The body of `format` is almost always one `format_to` call.**

Three ways to write one. Inherit from `formatter<string_view>` and you get width and alignment for free: the enum pattern. Write both members from scratch: the struct pattern, lab 2.3's TODO 3. Or store a spec in `parse` and forward it to a member's formatter: the one with its own spec, in the appendix.

The last row is the debugging guide. If `format("{}", t)` will not compile, the error says `formattable`, and it is one of three things: no specialization, `format` is not `const`, or `parse` is not `constexpr`.

The takeaway.

43. Formatting takeaway · target 1:19, ~1 min

`std::format` is `printf` with the format string checked by the compiler and user types allowed. `std::print` is `printf` ergonomics for it.

(pause)

Monday morning: write one `std::formatter` for your most-logged type, and delete the `to_string` helper it replaces.

Time check: 1:20, lab 2.3 now. After it, seven fast slides of library tour and the checklist. I'm going to move quickly; the tour is a catalog, and every slide has a demo file you can run later.

44. Lab 2.3: or this error, and formatting · target 1:20, ~10 min

Ten minutes, three TODOs. `expected` first, then one line of chaining, then a `formatter`. The third one is the at-home task 4 in miniature.

DO: Lab slide up, starter link in chat. One quiet minute.

What to watch for:

- TODO 1: forgetting `std::unexpected(...)` around the error. The compiler error is long because `expected` tries to build a `Reading` from a `ParseError`. Say once, to everyone: "Every error return needs `std::unexpected` around it."
- TODO 1: the success return is just `return Reading{...};`. No wrapping. People wrap it in `std::expected<...>{...}` and it works, but it is noise.
- TODO 2: `and_then(check_range)` works because `check_range` already returns an `expected`. If someone writes `transform(check_range)`, the result type becomes `expected<expected<...>>` and the error message is memorable. Let them read it, then say the rule: `transform` for functions that cannot fail, `and_then` for ones that can.
- TODO 3: the shape is in the comment; the stumble is the `format()` signature. Template on the context, `const`, return `std::format_to(ctx.out(), ...)`. If the `println` still will not compile after the specialization is in, the specialization is in the wrong namespace (it must be `std::formatter<Reading>`, at global scope with the `std::` qualifier or inside `namespace std`).

At seven minutes: solution link. Two minutes: show `parse_reading`'s new signature next to the old one, then the one-line `validated`, then the formatter.

> ASK: "Four output lines. Which stage produced each failure?" Expect: `temp,x` is the parser, the empty line is the parser, `temp,900` is `check_range`. "And the caller cannot tell the difference, and does not need to. One `if`, one `error()`."

45. C++17 tour, 1: `std::filesystem` · C++17 · target 1:30, ~1 min

Portable paths, directory iteration, and file queries, without a platform layer.

Line 11 on the right: `operator/` joins path components with the right separator. Line 12: `filename`, `extension`, `parent_path`. Line 15: `exists`. Line 16: iterate a directory. Line 20: `remove_all` with an `error_code`.

(beat)

Every operation has two overloads: one that throws, one that takes `std::error_code&` and never throws. If your codebase bans exceptions, use the second, always.

> DO: Terminal: `./build/demos/s02/demo_s02_tour_filesystem`. Expect `run1.csv .csv /tmp/telemetry`, then `exists: false`, and no directory listing, because the demo creates the directory but never the file. Recovery: none needed.
>
> IF BEHIND: `std::filesystem` gives portable paths and directory iteration, with a non-throwing `error_code` overload for everything.

46. `std::filesystem` at a glance · C++17 · target 1:31, ~1 min

One minute, the reference slide for `filesystem`. Paths join with `/` and come apart with `filename`, `stem`, `extension`, `parent_path`. Queries, metadata, create and remove, copy and rename, and the two iterators.

The row that is a habit: every operation has a throwing overload and an `error_code` overload. Pick the `error_code` one in anything that runs unattended. And treat `exists` as advice, not a guarantee: the file can vanish between the check and the open, so handle the open failing anyway.

47. C++17 tour, 2: small library additions · C++17 · target 1:32, ~1 min

The ones you'll actually use.

`clamp`, `gcd`, `lcm`. `invoke` calls anything callable the same way. `apply` unpacks a tuple into arguments.

The sleepers are lines 24 to 27: map `merge` and `extract`. `merge` splices nodes from one map into another without reallocating; anything with a key that already exists stays behind in the source. `extract` takes a node out, and you can **change its key in place** and put it back. Before C++17, changing a key meant erase and reinsert, with an allocation each way.

(beat)

`std::reduce` is an order-agnostic sum: preview for Session 4 and parallel algorithms.

> DO: Terminal: `./build/demos/s02/demo_s02_tour_cpp17`. Expect `100`, `6 12`, `5, 5, 2 1 1, 2 21`. Point at `2 1 1`: `x` stayed in `b` because `a` already had one. Recovery: read it aloud.
>
> IF BEHIND: `merge` and `extract` move map nodes without reallocating; the rest is small helpers.

48. C++20 tour, 1: `<bit>` and `source_location` · C++20 · target 1:33, ~1 min

Bit twiddling without compiler intrinsics, and logging without macros.

Line 17: `std::bit_cast<std::uint32_t>(f)`. The bytes of the float, reinterpreted. No undefined behavior, and it's `constexpr`. That replaces the `memcpy` idiom, the type-punning union, and the pointer cast, which is UB.

Then `popcount`, `has_single_bit` for "is a power of two", `bit_width`, `rotl`, and `std::endian::native`. These used to be `__builtin_popcount` and friends.

(beat)

And `source_location`: the `log` function at the top takes it as a default argument, so it captures the **caller's** file and line. **No more `__FILE__` and `__LINE__` macros in logging APIs.**

DO: Terminal: `./build/demos/s02/demo_s02_tour_bit`. Expect `3F800000`, `4`, `true`, `8`, `00000003`, `true`, and a log line with the file name, line 29, and the function name. Recovery: read it aloud.

IF BEHIND: `bit_cast` replaces type-punning safely; `source_location` as a default argument replaces logging macros.

49. C++20 tour, 2: containers and numerics · C++20 · target 1:34, ~1 min

Small things that each remove a line.

`std::erase_if` on any container: **the end of the erase-remove idiom.** `starts_with` and `ends_with`. `contains` on associative containers: no more `find() != end()`. `std::to_array` from a literal. `std::ssize`: signed size, which ends the `-Wsign-compare` warning in index loops. `midpoint`, which doesn't overflow the way `(a + b) / 2` does. And `std::numbers::pi`, so you can delete your `#define PI`.

(beat)

DO: Terminal: `./build/demos/s02/demo_s02_tour_cpp20`. Expect `1 3 5`, `true true`, `true`, `3 3`, `2 2.5`, `3.14159 1.41421`. Recovery: read it aloud.

IF BEHIND: `erase_if`, `contains`, `ssize`, `midpoint`, `numbers::pi`; each deletes an idiom.

50. C++20 tour, 3: <chrono> calendars and time zones · C++20 · target 1:35, ~1 min

The exercise's timestamp is a `long long`. It could have a type, and print itself.

Line 11: `sys_time<milliseconds>`: milliseconds since the epoch, as a type. Line 12: format it with `{:%F %T}`, no `strftime`. Line 14: truncate to a day. Line 15: `year_month_day`. Line 17: the weekday. Line 19: add a month, and the calendar arithmetic handles month lengths.

(beat)

Time zones need the tz database. libstdc++ 13 and later ship support; check your libc++ version.

DO: Terminal: `./build/demos/s02/demo_s02_tour_chrono`. Expect `2024-08-30 06:40:01.000 UTC`, `2024 Aug 30`, `Fri`, `2024-09-30`. Recovery: read it aloud.

IF AHEAD: Calendar arithmetic can produce invalid dates on purpose: January 31 plus one month is February 31, and `ymd.ok()` returns false. You decide whether to clamp or roll over, which is exactly the decision hand-written date code always got silently wrong.

IF BEHIND: `sys_time<milliseconds>` gives a raw timestamp a type and lets it format itself.

51. <chrono> at a glance · C++11, C++20 · target 1:36, ~1 min

One minute, `chrono` on one slide. The C++11 half: typed durations with literal suffixes, the three conversions that make rounding explicit, and the two clocks. **`steady_clock` to measure, `system_clock` to stamp; never the other way round.**

The C++20 half is what the previous slide showed: a time point with a unit instead of a `long long`, formatting with `%F %T`, calendar types that know how long February is, and time zones if your library ships the database.

`clock_cast`, in the last row, is what you reach for when `filesystem` hands you a `file_clock` time and you want to print it.

52. C++23 tour, 1: `flat_map`, `stacktrace`, `move_only_function` · C++23 · target 1:37, ~1 min

Three additions with uneven availability today.

`flat_map` is sorted vectors with a map interface: cache-friendly, cheap to iterate, expensive to insert. Right for a table built once and read many times, like the sensor table.

`std::stacktrace::current()`: "where am I", without a debugger. libstdc++ only today, and you link `-lstdc++exp`.

(beat)

`move_only_function`: a `std::function` that can hold a lambda capturing a `unique_ptr`. That matters for task queues, Session 5.

> DO: If you're on GCC: `./build/demos/s02/demo_s02_tour_stacktrace`; expect a few frames with `fail` and `main`. `flat_map`: Compiler Explorer tab 5, GCC trunk, expect `pressure 400`, `rpm 12000`, `temp_core 125`, then 3. Locally on GCC 14 you'll get the "not available" line, which is the gate working. Recovery: show the slide code and move on.
>
> IF BEHIND: `flat_map` for build-once tables, `stacktrace` for error paths, `move_only_function` for task queues; all gated on feature-test macros.

53. C++23 tour, 2: strings, bytes, and C APIs · C++23 · target 1:38, ~1 min

Odds and ends that each close a long-standing gap.

`std::string::contains`. Finally. `std::byteswap`: the endian conversion everybody has a macro for, now `constexpr`. And `resize_and_overwrite`: write directly into a string's uninitialized capacity, here with `snprintf`, and return the length you actually used. No zero-fill first.

(beat)

Not on the slide but in the demo file: `std::out_ptr`, which adapts a `unique_ptr` to a C API that fills in a `T**`. Every C library has that shape.

DO: Terminal: `./build/demos/s02/demo_s02_tour_cpp23`. Expect `true`, `3412`, `rpm=4811`, then on GCC 5 and 7 from `move_only_function` and `out_ptr`. Recovery: read it aloud.

IF BEHIND: `contains`, `byteswap`, `resize_and_overwrite`, and `out_ptr` for C APIs.

54. Deprecated and removed in the library · target 1:39, ~1 min

Fast. Things to grep for.

The big ones: `auto_ptr`, `random_shuffle`, `bind1st` are gone since C++17. `std::iterator` as a base class is deprecated; it's still there in C++23, but it warns. `std::aligned_storage` is deprecated in C++23. `<codecvt>` and `strstream` are removed in C++26.

(beat)

`aligned_storage` is the one that bites embedded code, in any hand-rolled small-buffer optimization. The replacement is `alignas(T) std::byte buf[sizeof(T)];`.

Library deprecation warnings come from `-Wdeprecated-declarations`, on by default, so under `-Werror` a compiler upgrade can turn these into build breaks. Clang-tidy's `modernize-` checks fix many of them.

IF BEHIND: Grep for `auto_ptr`, `random_shuffle`, `std::iterator`, and `aligned_storage`; the last becomes `alignas(T) std::byte buf[sizeof(T)]`.

55. Support matrix for this session · target 1:40, ~2 min

Every row on this slide was discovered building this course repo.

`expected`: fine on GCC 14, fine on Clang 18 with libc++, missing on Clang 18 with libstdc++. That's why the repo builds Clang with libc++. `from_chars` for double: missing on libc++ before 20. Range formatting, `flat_map`, `mdspan`: GCC 15. `stacktrace`: GCC only. `move_only_function` and `out_ptr`: newer libc++.

(pause)

The tool is the last line: `<version>` and the `__cpp_lib_` macros. Test the feature, not the compiler version. That's the recap slide coming back. The details are in `handouts/toolchain-support-matrix.md`.

ASK: What compiler is your production code on, and is it the one you develop with? (Expect: some say an older GCC for a certified or customer-mandated toolchain, sometimes years behind.) Say: "That's normal in our industry. So the question isn't 'is it in C++23', it's 'is it in our toolchain', and the macros answer that in code."

IF AHEAD: For older toolchains, the bridges are well known: `{fmt}` for `format` and `print`, `tl::expected` for `expected`, `gsl::span` or `tcb::span` for `span`. Their APIs are close enough that a later switch to `std::` is mostly a rename. The pattern: wrap them in your own namespace alias, so the switch is a one-line change.

> IF BEHIND: Support varies by library; `<version>` and `__cpp_lib_` macros are how code adapts.

The whole session on one page.

56. Interface design checklist · target 1:42, ~2 min

This is the screenshot slide. Everything on it is in today's solution.

Read-only text: `string_view` by value. Read-only contiguous elements: `span<const T>`. A buffer to write: `span<T>`. A value you keep: by value, then `std::move`. Maybe a result: `optional`. A result or a reason: `expected` with `[[nodiscard]]`. One of a fixed set: `variant`. One owner: `unique_ptr`. Borrowed and non-null: `T&`. Text output: `format` and `print` with a `formatter`.

(pause)

Take the screenshot now. Then apply it.

57. Lab 2.4: the checklist, applied · target 1:44, ~10 min

Ten minutes. Three C++11 signatures, each hiding something the type system could say. Rewrite each one using the checklist from the last slide, then fix the callers. No hints on the slide: the checklist is the hint.

> DO: Lab slide up, starter link in chat. Leave the checklist slide in your notes; people will ask you to go back to it. Go back to it once, for five seconds, then return to the lab slide.

What to watch for:

- `load_config`: the error enum is given in the TODO; people spend time on naming anyway. Nudge: "two values, `NotFound` and `BadRate`, the names are in the comment." The caller's `if (rc == 0)` becomes `if (auto cfg = load_config(path))`.
- `average`: two changes in one. `std::span<const double>` replaces the vector-plus-two-indices, and `std::optional<double>` replaces the `bool`-plus-out-param. The callers pass `all.subspan(2, 3)`; people write `std::span(samples).subspan(2, 5)` from habit (begin, end) and get the wrong average. The second argument is a count.
- `limits_for`: the easy one, if they remember that an `optional` can hold a struct. `return Limits{0, 12000};` converts implicitly.
- Anyone done early: ask them to write the fourth signature themselves: `void log_line(const char* fmt, ...)` to `std::print`.

At seven minutes: solution link. Put the three new signatures on screen next to the three old ones. That picture is the session.

> ASK: "For each one: which comment did the new signature delete?" Expect: "returns 0 or an error number", "false if the slice is empty", "lo and hi are only valid if true". Then: "Those comments were the interface. Now the types are."

58. Exercise: vocabulary types · target 1:54, ~1 min to launch

One minute. Open `exercises/s02-vocabulary-types/README.md` at home.

Tasks one to three are today's labs 2.2 and 2.3 applied to the real program: `expected` for the parser, `optional` for the lookups, `println` for the report. Then tasks four to nine. **The test `s02_report_identical` diffs your report against the starter's. One space off and it fails. That's the point: the behavior must not change.**

(beat)

No toolchain at home? The README has Compiler Explorer links for the whole program, starter and solution. The solution is next week's starter, so finishing is optional, but reading it is not.

59. Session 2 takeaway · target 1:55, ~2 min

A modern interface says what it means. `optional` means "maybe". `expected` means "or this error". `string_view` means "I'll only look". `span` means "a contiguous run I don't own".

(pause)

The information that used to live in comments, sentinels, and out-parameters now lives in the signature, where the compiler can see it.

Next session is compile time. The CRC table, the configuration table, and the `serialize` overload set are the targets: `constexpr`, `consteval`, concepts, and deducing `this`. Today's solution is next session's starter, so if you don't finish tasks 4 to 9, take ours.

The support matrix and the feature timeline handouts are updated for today. Thanks, everyone.

Appendix slides (in the PDF, not presented)

These sections are kept for the `IF AHEAD` moments and for answering questions; the slides sit after the Further reading divider (deck slide 60).

A1. `string_view` operations · C++17 (deck slide 61)

Everything you can do without owning the characters, and all of it is O(1) or a search.

Walk the `ops` function on the right with me. Line 31 on the right: `line.substr(0, 5)`. On a `std::string`, `substr` allocates a new string. On a `string_view` it returns a smaller view. Constant time, no copy.

Line 32: `remove_prefix(2)`. This mutates the view itself, not the characters. It just moves the start pointer forward. That's how a hand-written tokenizer walks a line: chop off the front, look again.

Lines 33 and 34: `starts_with` is C++20, `contains` is C++23. And those exist on `std::string` too now, so the `find(...) != npos` and `compare(0, n, ...)` idioms can go.

(beat)

Line 35: `find`, exactly like `std::string`.

> DO: The output you just ran was `1725, 1 1 2`. Point at it: `head` is `1725,`; after dropping two characters the view starts at `25,temp...`, so `starts_with("25")` is 1, `contains("temp")` is 1, and the first comma is at index 2. Point at the `printf` on line 36: `%.*s` with an explicit length. That's how you print a view with C I/O, because, spoiler for the next appendix slide, there's no NUL at the end.
>
> IF AHEAD: `substr` on a view still bounds-checks `pos`: if `pos > size()` it throws `std::out_of_range`. So it's not a free-for-all. Meanwhile `remove_prefix(n)` with `n > size()` is undefined behavior, no check. In review, any `remove_prefix` should have a size check right above it or an obvious invariant.
>
> IF BEHIND: `substr` and `remove_prefix` on a view are O(1) and copy nothing; `starts_with`, `ends_with`, and `contains` are C++20 and C++23 conveniences.

Now the price of all this.

A2. A `string_view` is not a C string · C++17 (deck slide 62)

A `string_view` has no NUL terminator. Anything that walks to a NUL reads past the end.

Look at `bad` on the right, line 11. `std::strtod(field.data(), nullptr)`. `strtod` doesn't know there's a length. It reads until it finds something that isn't part of a number. `field.data()` points into the middle of somebody else's buffer.

In the demo, `field` is the first five characters of `"41.25,99"`. So `bad` "works": `strtod` reads `41.25`, hits the comma, stops. That's luck.

(pause)

Change the data so the next character is a digit, say the field is `"41.2"` and the buffer says `"41.25"`, and `bad` returns 41.25 instead of 41.2. Wrong answer, no crash, no warning. And if the field is at the very end of a buffer that isn't NUL-terminated, a packet from a socket, a memory-mapped file, a DMA region, it reads past the end of the buffer.

The fix is `ok`, line 15: copy into a `std::string` and call `c_str()`. Yes, that's an allocation. That's the honest cost of calling a C API that wants a NUL.

The exercise hits this exactly once: the `strtod` fallback under `#ifndef __cpp_lib_to_chars` has to copy the field. That's the libc++ path from the recap. It's the one place in the parser that still allocates.

> DO: Terminal: `./build/demos/s02/demo_s02_string_view_not_cstring`. Expect `41.250000 41.250000`. Say: "Both right. That's the dangerous part."

Optional live edit: on line 21 change `substr(0, 5)` to `substr(0, 4)`, rebuild, rerun: `bad` still prints 41.25, `ok` prints 41.2. Recovery: if the rebuild is slow, describe it and move on.

IF AHEAD: This is why `from_chars` is the right tool with views: it takes a begin and an end pointer, so it can't read past the end. Same for `std::string_view::find` versus `strchr`. A review heuristic: grep for `.data()` on anything that's a `string_view`. Every hit is either passed with a length, `%.*s` style, or it's a bug. In a MISRA or AUTOSAR shop, this is the same class of finding as unbounded `strlen` on external data.

IF BEHIND: A view has no NUL; any C API that needs one gets `std::string(sv).c_str()`, and that's the one copy the exercise keeps.

So when do you still take a `std::string`?

A3. span mechanics · C++20 (deck slide 63)

Six bullets, quickly.

Dynamic extent: `std::span<T>` is a pointer and a size, sixteen bytes. Static extent: `std::span<T, 8>` is just a pointer, because the size is in the type. **Static extent is the embedded case: the size is checked at compile time when you build the span from an array.** A function that takes exactly an eight-byte header can say so, and passing a twelve-byte array doesn't compile.

`span<const T>` reads; `span<T>` writes through. The constness of the elements is in the template argument, not on the span.

`first(n)`, `last(n)`, `subspan(offset, n)`: slicing without copying.

`as_bytes` and `as_writable_bytes`: reinterpret the same memory as a span of `std::byte`. Next slide.

(beat)

No bounds checking on `operator[]`. Like a raw pointer. And there's no `.at()` until C++26. If you want checking today, you turn on your library's hardening mode, libstdc++'s `_GLIBCXX_ASSERTIONS` or libc++'s hardening modes, in debug and test builds.

And the same lifetime rule as `string_view`. **A parameter, yes. A member, only if you know who owns the memory.**

IF AHEAD: Constness gotcha: a `const std::span<int>` still lets you write the ints. The span is const, the elements aren't. It's a pointer, so `int* const` versus `const int*`. In review, if a function only reads, the parameter must say `span<const T>`, and the `const` on the span itself means nothing. Second gotcha: `span<T>` converts implicitly to `span<const T>`, and dynamic to static extent needs an explicit construction, so the conversions go in the safe direction only.

IF BEHIND: Static extent puts the size in the type; `span<const T>` is the read-only one; no bounds checks; same lifetime rule as `string_view`.

Here's where this pays off for us.

A4. `std::mdspan`: a preview · C++23 (deck slide 64)

One slide, for anyone who has image buffers, sensor grids, or matrices.

`mdspan` is `span` with a shape: a multidimensional view over the same flat memory, with no copy.

Line 2 of the code: a 3 by 4 view over a flat array of twelve floats. Line 3: `m[1, 2]`. That's the C++23 multidimensional subscript operator, a comma inside the brackets, which used to be the comma operator and is now a real multi-argument `operator[]`. Line 4: the same thing with the extents decided at run time.

(beat)

The interesting part is the layout policy. `layout_right` is row-major, C order. `layout_left` is column-major, Fortran and MATLAB order. `layout_stride` is arbitrary strides. Same buffer, different interpretation, no copy. If you've ever written `buf[row * cols + col]` by hand and gotten the order wrong once, this is for you.

Availability: libc++ has it, which is our Clang build; libstdc++ gets it in GCC 15.

> DO: Compiler Explorer tab 4: the slide snippet, Clang 18 with `-stdlib=libc++ -std=c++23`, or GCC trunk. Add `std::println("{}", buf[6]);` (or a `printf`) and show 7: row 1, column 2 in a 4-wide row is index 6. Recovery: if the tab misbehaves, skip it; this slide is a preview.
>
> IF BEHIND: `mdspan` is a shaped view over flat memory with row-major or column-major layout; GCC 15 or libc++ today.

Back to strings, and a lookup that allocates when it shouldn't.

A5. Transparent comparators · C++14 (deck slide 65)

Looking up a `string_view` key in a `map<string, T>` used to construct a `std::string` per lookup.

Look at the `plain` map on the right, line 10. `std::map<std::string, int>`. In `lookups`, line 14, to search it with a view you have to build a `std::string`. That's a construction every lookup, and for any key longer than the small-string buffer, a heap allocation. In a hot path that's real.

Line 11: the same map with a third template argument, `std::less<>`. Empty angle brackets. That's the C++14 "transparent" comparator: it compares any two things that can be compared with <. Because it's transparent, `find` gets a template overload that takes any key type. Line 15: `transparent.find(key)` compares the view against the stored strings directly. No temporary.

(pause)

Add `std::less<>` to every `std::map` and `std::set` keyed by `std::string`. It costs nothing.

For `unordered_map` you need C++20, and you need both a transparent hash and a transparent equality, `std::equal_to<>`. The slide mentions just the hash; you need both, or the heterogeneous `find` overload doesn't appear.

In the exercise, at-home task 9: `StatsBySensor` gets `std::less<>` so that `find("rpm")` works from a literal.

> DO: Terminal: `./build/demos/s02/demo_s02_transparent_compare`. Expect 1. Not exciting; the point is it compiled. Optional: on Compiler Explorer, change `transparent.find(key)` to `plain.find(key)` and show the error: no conversion from `string_view` to `const std::string&`. Recovery: skip the edit.
>
> ASK: Why isn't `std::less<>` the default for `std::map<std::string, T>`? (Expect: backward compatibility. Making it transparent changes overload resolution for existing code, and some key types have conversions that would behave differently.) Say: "Right. It's opt-in because the default was fixed in 1998."
>
> IF AHEAD: For `unordered_map`, the transparent hash is a small struct: `struct SvHash { using is_transparent = void; std::size_t operator()(std::string_view s) const { return std::hash<std::string_view>{}(s); } };`, then `std::unordered_map<std::string, int, SvHash, std::equal_to<>>`. The `is_transparent` typedef is the magic marker in both cases; that's literally what the library checks for.
>
> IF BEHIND: `std::less<>` as the third template argument lets `find` take a `string_view` with no temporary string.

That's the views segment.

A6. The pointer-to-member trap · C++23 (deck slide 66)

The obvious way to write that units lookup is: `find_sensor(name).transform(&SensorConfig::units)`. A pointer to member as the projection. Ranges algorithms accept that happily. It doesn't compile here, and the error is a wall of text.

Why? Invoking a pointer to data member on an object gives you a **reference** to that member, not a copy. On a temporary optional, it's an rvalue reference, `string_view&&`. On a named optional, it's an lvalue reference, `string_view&`. `transform` wants to build an `optional` of whatever the function returns.

(pause)

And `optional` of a reference type is ill-formed in C++23. So it fails either way. The slide says "on a temporary", but I checked: GCC and Clang reject it on a named optional too. It's not about temporaries; it's about references.

The fix is the lambda you saw: `[](const SensorConfig& c) { return c.units; }`. That returns by value, so `transform` builds an `optional<string_view>`.

> DO: Compiler Explorer tab 2: `optional_monadic.cpp` with the trap uncommented, GCC 14, `-std=c++23`. Expect an error from inside `<optional>`: a static assertion failing on `is_object_v<...&>`, or "union member has reference type". Scroll to the first line that mentions your file, and show that the actual cause, "a reference type", is buried three screens down. Recovery: if the tab is broken, read the README's task 8 line instead: "read the compiler error once".

Read the error once, so the next time you see it, you know it in five seconds. That's why the exercise asks you to try it.

> IF AHEAD: This is a value-category bug, and the supplemental deck on value categories explains the general rule: `std::invoke` with a pointer-to-data-member on an xvalue yields an xvalue of the member. C++26 adds `optional<T&>`, so the lvalue case may start compiling there, giving you an optional that refers into the original. Which may be what you want, or may be a dangling reference. Another reason to prefer the explicit lambda: it says "copy" out loud.
>
> IF BEHIND: `transform(&T::member)` doesn't compile because it yields a reference and `optional<T&>` doesn't exist in C++23; use a lambda that returns by value.

From "maybe one" to "exactly one of several".

A7. `variant` details worth knowing · C++17 (deck slide 67)

Fast slide. Six bullets, one sentence each.

A variant default-constructs its **first** alternative. **If "empty" is a legitimate state, put `std::monostate` first.** That's the one that comes up in practice.

`valueless_by_exception()`: true only if an exception was thrown while switching alternatives, after the old one was destroyed. Rare. `visit` throws `bad_variant_access` on it.

Comparison and hashing work if every alternative supports them. Alternatives may repeat, `variant<int, int>`, and then you use `get<0>`, not `get<int>`. `visit` with two variants dispatches on both: double dispatch for free. And C++26 adds member `visit`.

(beat)

> IF BEHIND: Use `std::monostate` first when "empty" is a state; the rest is reference material.

And the type for when you can't name the alternatives.

A8. Where these came from (deck slide 68)

This is the optional history slide. If you're behind, skip it.

These types were in production use in Boost for over a decade before they were standardized. Boost.Optional shipped in 2003, Boost.Variant very shortly after. Optional nearly made C++14 and was pulled into a technical specification in 2013. Variant took longest because of one question: what state is the variant in if assigning a new alternative throws? The answer, "valueless by exception", settled in 2016, and `optional`, `variant`, and `any` all landed in C++17.

`expected` followed the same path: Boost.Outcome and `tl::expected` in the field, adopted for C++23 in 2022.

(beat)

The point: the designs are not experimental. Adopting them is low risk.

> IF BEHIND: Skip the slide: "These were proven in Boost for over a decade; the designs aren't experimental."

The segment in three lines.

A9. `expected<void, E>` · C++23 (deck slide 69)

"Did it work, and if not, why?" No value to return, but a reason to report.

`write`, line 12 on the right. Returns `expected<void, IoError>`. Failure: `std::unexpected(IoError::NotOpen)`. Success, line 15: `return {};`. An empty brace is the success value.

Line 19: `if (auto r = write(...); !r)`, the C++17 if-with-initializer. Test, then `r.error()`.

(beat)

This replaces `bool write(...)` plus a global or thread-local "last error". And this is the one where `[[nodiscard]]` on the function matters most, because there's no value to tempt the caller into checking.

The slide mentions coroutines. That's a Session 5 preview, and to be precise: the standard doesn't make `expected` awaitable. You'd write that adapter yourself. It pairs naturally; it's not built in.

> DO: Terminal: `./build/demos/s02/demo_s02_expected_void`. Expect a single `0`: the `"hello"` write succeeded and printed nothing, and `write("x", false).has_value()` is 0. Recovery: read it aloud.
>
> IF BEHIND: `expected<void, E>` is success-or-reason; `return {};` is success; mark it `[[nodiscard]]`.

The segment, in one sentence.

A10. A formatter with its own spec · C++20 (deck slide 70)

`parse` sees the text between the colon and the closing brace. It can mean whatever your type needs.

`Celsius`, line 12. One member, `fahrenheit`, defaults to false.

`parse`, line 15. Look at it character by character. Line 16: start at the beginning of the spec. Line 17: if the first character is `f`, set the flag and step past it. Line 18: if we're not at the closing brace now, throw `std::format_error`. Line 19: return where we stopped.

(pause)

`format`, line 22: reads the flag and formats in Celsius or Fahrenheit.

Here's the subtle part. **`parse` runs at compile time when the format string is a literal, so a bad spec is a compile error too.** That `throw` on line 18 isn't a run-time exception in normal use. Throwing during constant evaluation makes the expression not a constant, and that becomes a build failure pointing at your format string. Your custom spec gets the same compile-time checking as the built-in ones.

> DO: Terminal: `./build/demos/s02/demo_s02_formatter_spec`. Expect `41.2C 106.2F`. If someone asks why 41.25 printed as 41.2: 41.25 is exactly representable in binary, so it's an exact tie, and the formatting rounds ties to even. Optional live edit: change `{:f}` on line 29 to `{:k}` and rebuild; expect a compile error mentioning the spec. Undo. Recovery: skip the edit.
>
> IF AHEAD: To combine your own spec with width and alignment, hold a nested `std::formatter<double>` member, let it parse the rest of the spec, and forward to it in `format`. That's the "forwarding to a nested formatter" pattern. And the round-half-even result above is the kind of thing that fails a byte-for-byte diff test if you change from `%.1f` to something else in the middle of a migration. `printf` and `format` agree on it, which is why the exercise's diff test can pass.
>
> IF BEHIND: `parse` reads your custom spec, stores it in members, and a bad spec is a compile error.

Now containers.

A11. Formatting ranges · C++23 (deck slide 71)

Printing a vector used to be a loop. Now it's `"{}"`.

Line 16: the vector prints as `[1.5, 2.25, 3]`. Line 17: `{::.1f}`, two colons. The spec after the second colon applies to each element. Line 18: `{:n}` drops the brackets. Line 19: a map prints with braces, keys and values, and strings come out quoted. Line 20: a pair prints in parentheses.

(beat)

Availability: libc++ has it, GCC gets it in 15. **Gate it on `__cpp_lib_format_ranges`**, which is exactly what this demo does.

DO: Terminal: `./build-clang/demos/s02/demo_s02_format_ranges` if you have the Clang build. Expect the five lines from the comments, with `2.2` from `2.25` (tie, rounds to even). On the GCC 14 build, expect `range formatting not available in this standard library`; say "that's the feature-test macro working", and switch to Compiler Explorer tab 6. Recovery: read the comments.

IF BEHIND: `println("{}", vec)` works in C++23; `{::spec}` formats each element; GCC 15 or libc++.

And for embedded: no allocation.

A12. Formatting without allocating · C++20 (deck slide 72)

Embedded and hot paths: format into a buffer you already own.

Line 12: a 32-character `std::array` on the stack. Line 13: `format_to_n` writes at most `buf.size()` characters. **It never overruns.** It returns a struct with `out`, where it stopped writing, and `size`, which is how much it **would** have written without the limit.

(pause)

Careful with line 14. The variable is called `written`, but it holds `r.size`, the untruncated size. Here it fits, so they're equal. If the output were truncated, `written` would be bigger than the buffer, and the `printf` with `%.*s` at line 25 would read past the end of `buf`. The count of characters actually written is `r.out - buf.data()`. That's a review finding I'd want caught. Also, `format_to_n` doesn't add a NUL.

Line 15: `formatted_size` tells you how big the output would be, so you can size a buffer first.

Lines 18 and 19: `format_to` with `back_inserter` appends to any output iterator.

Lines 21 to 23: a format string from a variable. That's `vformat`, and it loses the compile-time check. Line 22: `make_format_args` wants lvalues, which is a C++23 defect fix, so a named variable, not a literal. C++26 adds `std::runtime_format` to say "this is a runtime string" explicitly.

DO: Terminal: `./build/demos/s02/demo_s02_format_to_buffer`. Expect `rpm:4811.00 11 11 1 2      42`. Point: 11 characters written, 11 needed, the appended string `1 2`, and 42 right-aligned in 6. Recovery: read it aloud.

IF AHEAD: Honest caveat for embedded: `format_to_n` avoids heap allocation for the output, but `std::format` is a big library. Floating-point formatting pulls in a lot of code, and binary size on a microcontroller can grow noticeably. Measure with your linker map before you swap every `snprintf` on a small target. On a Linux-class processor, it's a non-issue.

IF BEHIND: `format_to_n` into a fixed buffer never overruns; use `r.out`, not `r.size`, for what was written.

How do you get a big codebase there?

A13. Migration (deck slide 73)

The table is mechanical. Drop `c_str()` and the casts: `%lu` and `%zu` are just `{}`. Width and alignment move after the colon. `%08.3f` becomes `{:08.3f}`, same order. `std::hex` becomes `{:x}`, with no sticky state. `ostringstream` becomes `std::format`. `snprintf` becomes `format_to_n`.

`clang-tidy`'s `modernize-use-std-print` does the `printf` rows automatically.

(beat)

In-class task 3 is this table applied to `report.cpp`. And the diff test catches one space of difference.

> IF AHEAD: One more trap for embedded code: `iostream` prints `uint8_t` as a character, because it's `unsigned char`. `std::format` prints `uint8_t` as a number, because only `char` is treated as a character. That difference is usually a fix, but it will change output byte-for-byte, so a migration that touches logging needs its golden files regenerated deliberately.
>
> IF BEHIND: The translation is mechanical, and `modernize-use-std-print` automates the `printf` cases.

The segment in two sentences.

Running the labs

The per-lab coaching is in the script at slides 14, 27, 44 and 57. The rhythm is the same every time:

1. Lab slide up, starter link in chat, one quiet minute while people open the file. Then stay quiet for five more; answer chat, do not lecture.
2. At seven minutes, paste the solution link and walk the three changes in two minutes. The debrief is the last word on the block's topic, so it is scripted.
3. One question to the room, then move on at ten minutes whether or not people are done. Nothing later depends on finishing a lab.

If the lecture runs late, the lab still starts within two minutes of its target; cut the slide before the takeaway instead. If a lab runs over because the room is engaged, take the time out of block 4's library tour, which is the only part of the deck built to be skimmed.

The at-home exercise

Launched at slide 58 in one minute; nothing is done in class. What to say when it comes up in chat during the week, and in next session's recap:

- Task 1: returning `ParseError::Empty` without `std::unexpected`. The error is long, because `expected` tries to convert a `ParseError` into a `Record`. "Every error return needs `std::unexpected(...)` around it." (Lab 2.3 TODO 1 was the rehearsal.)

- Task 1: deleting `ParseError::None` breaks other code: a `switch` with a `None` case, a `to_string`, tests that compare against `None`. That is the compiler showing every place that relied on the fake value. Fix each one; do not add it back.
- Task 2: `if (cfg == nullptr)` stops compiling. An optional compares against `std::nullopt`, or better, just `if (!cfg)`. Call sites using `cfg->` keep working.
- Task 3: leaving `\n` at the end of the format string with `println`. Double newline, and the diff test fails. The most common failure on this task.
- Task 3: `%-16s` becomes `{:16}` (strings left-align by default) but `%5s` must become `{:>5}` (`printf` right-aligns).
- Task 4 (`formatter<Status>`): inherit from `formatter<string_view>` and forward; Lab 2.3 TODO 3 wrote the from-scratch version.

Next session's recap slide should pick up whichever of these the chat shows most.

Likely questions and answers

Q: Our certified toolchain is GCC 11 (or older). What can we use? A: Everything from C++17 today: `string_view`, `optional`, `variant`, `any`, `filesystem`, and `span` is there from GCC 10. `std::format` needs GCC 13, `std::expected` GCC 12, and `std::print` GCC 14, so on GCC 11 use `{fmt}` and `tl::expected` behind a namespace alias. Check `handouts/toolchain-support-matrix.md` and confirm with the `__cpp_lib_` macros on your actual toolchain.

Q: Does returning `optional<SensorConfig>` copy the struct where the pointer didn't? A: Yes, it returns a copy, held inline. For small structs that's cheaper than the indirection and removes the lifetime question. For large objects, a non-owning `const T*` is still the right return, or `optional<T&>` once you're on C++26.

Q: Is it safe to put `string_view`, `span`, `optional`, or `expected` in an ABI boundary, like a shared library interface? A: Within one toolchain and standard library, yes, and `string_view` and `span` pass in registers as two words on x86-64 and AArch64. Across different standard libraries (libstdc++ versus libc++) the layouts differ, so never put `std::` types in a C ABI or a binary plugin interface built by a different toolchain. At a C boundary, pass pointer and length and wrap it into a span on each side.

Q: How do `expected` and `optional::value()` behave with `-fno-exceptions`? A: They work, and everything except `value()` behaves normally. `value()` on an empty or error state can't throw, so the library aborts instead (libstdc++ calls `abort()`). In a no-exceptions codebase, test explicitly and treat `value()` as an assertion.

Q: What do MISRA C++ or AUTOSAR say about these types? A: AUTOSAR C++14 predates them. MISRA C++:2023 targets C++17, so `string_view`, `optional`, and `variant` are within its language; I'd check your specific rule set before assuming any particular rule, especially around dynamic memory, which affects `any` and `std::string`. `std::expected` is C++23, so it's outside both today; teams usually justify it by deviation or use a vetted in-house equivalent.

Q: Is `std::format` slower or bigger than `printf`? A: Run time is usually comparable or faster; `{fmt}`'s published benchmarks beat `printf` in many cases. Compile time and binary

size are larger, especially with floating point. On a microcontroller, measure with a linker map; on Linux-class hardware it's a non-issue.

Q: Is `std::expected` `[[nodiscard]]`? A: Not by the standard. A discarded call compiles silently unless the function is marked `[[nodiscard]]`. So mark every `expected`-returning function; the exercise solution does.

Q: What happens if someone stores a `string_view` from `split` after the line buffer is reused? A: It dangles, silently, and the views see the next line's characters or freed memory. `split` returning views is correct only because the caller uses them before the next `getline`. That's the "write the lifetime down" rule: a comment on `split` saying the views are valid until the buffer changes.

Q: Variant or virtual for our message handlers? A: If the set of message types is fixed by a protocol, variant: exhaustive, no heap, value semantics. If other teams or plugins add types you can't list, virtual. Many systems use both: virtual for drivers, variant for the messages they produce.

Q: Does `std::visit` cost more than a `switch`? A: For small variants, GCC and Clang generate a jump table or switch, comparable to a hand-written one. Older libstdc++ versions generated a function-pointer table, which inlines less well. If it's in a hot loop, look at the assembly on Compiler Explorer.

Q: Can we use `std::print` from multiple threads? A: Each `print` call writes its formatted output in one operation, so lines from different threads don't interleave within a call the way chained << operations do. Ordering between threads is still up to you. If you need guaranteed atomic log lines, format into a string first and write it once, or use a logging library.

Deck issues found

Slide numbers in this list are the deck's numbering before the block restructure (see `docs/session-format.md`); the items are kept for the record.

Status: fixed in the deck, demos, outline and README on 2026-10-02 (see scripts/README.md). Items kept for the record. Still open: the Compiler Explorer `<add short link>` placeholders in every demo file header.

- Slide 21 (also exercise README task 8 and the toolchain handout): the pointer-to-member trap is not specific to temporaries. `opt.transform(&SensorConfig::units)` on a named optional also fails (it yields `string_view&`, and `optional<string_view&>` is ill-formed in C++23). Verified on GCC 13 and Clang 18. The script says this.
- Slides 23 and 24: the `overloaded` struct with no deduction guide needs C++20 aggregate CTAD; under `-std=c++17` it fails to compile (verified on GCC 13 and Clang 18), yet the badge says C++17. Add `template <class... Fs> overloaded(Fs...) -> overloaded<Fs...>;` or note the C++20 requirement.

- Slide 23: "Add a fourth alternative ... stops compiling" overstates it. An alternative that converts implicitly to an existing lambda parameter (`float` to `double`, `const char*` to `std::string`) compiles silently.
- Slide 30: "Ignorable? no" for `optional` and `expected` isn't right. Neither type is `[[nodiscard]]` in the standard; a discarded call compiles silently. The demo comment on slide 31 (`expected_basics.cpp` line 23, "`[[nodiscard]]` by nature") has the same problem. Slide 34 correctly says you need the attribute.
- Slide 41: "`println` with no arguments prints a newline" is C++26 (P3142), not C++23.
- Slide 45 / `format_to_buffer.cpp`: `written` is set from `r.size`, the untruncated size. Using it as the `%.*s` length on line 25 over-reads `buf` if output is truncated. Use `r.out - buf.data()`, and rename the variable.
- Slide 55: `std::iterator` was not removed in C++20; it is still deprecated in C++23 (libstdc++ 13 only warns). `<codecvt>` and `strstream` are removed in C++26 (P2871, P2867), so they belong in the Removed column. "`<ciso646>`'s point" is unclear; `<ciso646>` was removed in C++20, not deprecated. Library deprecation warnings come from `-Wdeprecated-declarations` (on by default), not `-Wdeprecated`.
- Slide 55 notes: `sizeof T` should be `sizeof(T)`.
- Slide 53 vs handout: the slide says libc++ 20 for `flat_map` and libc++ 19 for `move_only_function`; the handout and `tour_flat_map.cpp` say libc++ 20 has `flat_set` only, and `tour_cpp23.cpp` says libc++ has no `move_only_function` yet. Reconcile.
- Slide 54: the notes describe `out_ptr`, which is not on the slide (it is only in `tour_cpp23.cpp`). The "Also" line repeats `resize_and_overwrite`, which the code already shows.
- Slide 15: C++20 heterogeneous lookup for `unordered_map` needs a transparent hash and a transparent key-equal (`std::equal_to<>`); the slide mentions only the hash. "An allocation per lookup" holds only for keys longer than the SSO buffer (not "`rpm`").
- Slide 8: item (2) `return s.substr(0, 5);` returns a view of a temporary `std::string` made by `substr`, not of the local `s` (Clang says "local temporary object"). To show "view of a local", use `return s;` or `std::string_view(s).substr(0, 5)`.
- Slide 28: I believe Boost.Variant first shipped in Boost 1.31 (early 2004), not 2002 to 2003. Please verify.
- Timing: the Exercise and close segment starts at 1:40 and must hold the checklist, the launch, a "20 minute" exercise, a debrief, and the takeaway in 20 minutes. Only about 16 minutes of exercise fit. The outline's "58 content slides for 90 minutes" doesn't match the deck either: it has 59 slides and 100 minutes before the exercise segment.
- Slide 48 / `tour_filesystem.cpp`: the demo never creates `run1.csv`, so `exists` prints false and the directory loop prints nothing.
- Slide 36 / `expected_void.cpp`: uses `std::string_view` but includes only `<string>`. It works in practice, but `<string_view>` should be included.